

Оглавление

Оглавление.....	6
Введение.....	8
1. Обзор	10
1.1. Определение	10
1.2. Системы для управления предприятием - CRM, ERP	10
1.3. Актуальность применения готовых CRM, ERP систем на малом предприятии	13
2. Анализ	14
2.1. Постановка задачи.....	14
2.2. Требования к ПО	14
2.2.1. Бизнес-требования.....	14
2.2.2. Требования безопасности	14
2.2.3. Нефункциональные требования.....	14
2.2.4. Качественные характеристики	15
2.2.5. Требования к интерфейсу	15
2.2.6. Основные функции системы в целом:	15
2.3. Группы пользователей	15
2.4. Системные требования	16
2.5. Сведения о предприятии.....	17
3. Проектирование.....	18
3.1. Компонентная структура ПО	18
3.2. Структура данных	18
3.3. Взаимодействие клиента и сервера	21
3.4. Структура клиентского приложения	22
3.5. Экранные формы	23
4. Разработка приложения	29
4.1. Методология разработки	29
4.2. Инструменты разработки	29
4.2.1. Языки программирования	29
4.2.2. Библиотеки и фреймворки.....	30
4.2.3. Программы и сервисы.....	30
4.3. Разработка классов.....	31
4.3.1. FeedbackPage – Обратная связь.....	32
4.3.2. WorkManagement – Учет рабочего времени	34
4.3.3. Clients – Клиенты	37
4.3.4. Requests – Заявки на производство	39
5. Краткое руководство пользователя	43
6. Тестирование	46
6.1. Выбор видов тестирования.....	46

6.2. Результаты тестирования.....	46
6.3. Примеры созданных тестов.....	46
Заключение	48
Список Литературы.....	49
Приложение	50

Введение

В современном мире функционирование экономически эффективного предприятия подразумевает автоматизацию процессов, с помощью которых появляется возможность проводить многоуровневый анализ множества факторов, которые непосредственно влияют на производительность и эффективность производства. Появившиеся в последние десятилетия информационные технологии предоставляют новые методы и пути управления, контроля и анализа работы предприятий и расширяют возможности даже для малых предприятий.

Реалии нынешней рыночной экономики и высокая конкуренция, в особенности для малого предприятия, требуют максимальной экономической эффективности производства, анализа существующего рынка и прогнозирования на основе как рыночных, так и собственных данных о продукции. Для достижения данных целей необходимо грамотно отобрать учитываемые при анализе данных параметры, которые позволят производству быть максимально конкурентоспособным. Такими параметрами могут быть как учет рабочего времени, аналитика клиентов производства, так и обратная связь с сотрудниками и заявки на производство. Именно автоматизация этих задач позволит более рационально рассчитывать свои ресурсы, грамотно управлять персоналом и планировать свои действия на рынке.

Многие предприятия на рынке используют для выполнения вышеописанных задач CRM (Customer Relationship Management) или ERP (Enterprise Resource Planning) системы. Это программное обеспечение для управления бизнес-процессами и автоматизации определенных функций производства. По сути эти системы схожи между собой – они позволяют производству автоматизировать определенные бизнес-процессы, но области их применения отличаются. В то время как CRM-системы обеспечивают качественное и эффективное взаимодействие с клиентами и их заказами, ERP-системы направлены на эффективное управление всеми основными подразделениями предприятия. Для предприятий, занимающихся не только производством продукции, но также и ее реализацией, грамотным решением будет рассматривать возможность интеграции этих систем для увеличения эффективности производства, продаж и конкурентоспособности на рынке в своей сфере.

Но при выборе системы управления на малом предприятии возникает ряд проблем. Рассмотрим, к примеру – 1С:ERP: высокая стоимость готовой системы (даже самая базовая версия стоит больше 420 тыс. руб.), необходимость дополнительного обучения персонала, частые обновления системы, требуется наличие штатного программиста 1С, низкая скорость работы. Также на малом предприятии часто отсутствует заранее написанное техническое задание, и необходимые требования могут появляться по мере

развития самого производства, что усложняет процесс разработки и проектирования.

Анализ негативных факторов при интеграции готовой коммерческой системы в производственные процессы малого предприятия показывает, что актуальным становится создание частной и специализированной системы для конкретных нужд предприятия. Именно такая система позволит малому предприятию эффективно автоматизировать необходимые бизнес-процессы, удовлетворяя меняющиеся требования, в условиях специфики производства и торговли продукцией.

Сейчас на рынке существует огромное количество предприятий, которые только начинают производство, и даже торговлю своей продукцией, для которых приобретение и разработка готовых коммерческих систем для управления не представляется возможным либо релевантным. Поэтому частные CRM или ERP-системы играют большую роль в экономической эффективности, грамотном управлении персоналом и ресурсами малого предприятия.

Данная дипломная работа посвящена проектированию и разработке одного из основных компонентов системы по управлению малым предприятием - клиентского приложения, предназначенного для использования непосредственно сотрудниками предприятия.

1. Обзор

1.1. Определение

Малым предприятием называется предприятие, которое имеет численность персонала от 16 до 100 включительно [2], предельный доход до 800 млн. рублей [2] и также доля участия, принадлежащая одному или нескольким юридическим лицам, не являющимся субъектами малого и среднего предпринимательства, не должна превышать также 25% [2].

Доля малого бизнеса в России приблизительно равна 20%, среди них примерно 250 тыс. предприятий юридических лиц и 30 тыс. предприятий индивидуальных предпринимателей. Таким образом, каждое пятое предприятие является малым, что говорит о высокой конкурентности на этом уровне.

1.2. Системы для управления предприятием - CRM, ERP

CRM – это система управления отношениями с клиентами. Она упрощает процесс взаимодействия с клиентами, сохраняя всю необходимую информацию о них, ведя учет всех контактах, сделках и заявках.

ERP – это система планирования ресурсов компании. Данная система позволяет вести учет всех подразделений предприятия, систематизируя и автоматизируя множество бизнес-процессов взаимодействия.

Задачи этих систем довольно близки – автоматизация бизнес-процессов предприятия. Обе системы позволяют хранить и обрабатывать данные с целью повышения экономической эффективности предприятия, но их область применения несколько отличается. CRM-системы применяются в сфере продаж, работы с клиентами и заказчиками. Это своего рода органайзер для обеспечения реализации всех необходимых задач по взаимодействию с клиентами в заданные сроки и с высоким качеством. Включает в себя как учет базы клиентов, планирование действий, контроль оплаты, оформление заявок, так и в некоторых случаях – запись звонков. В то время как область применения ERP-систем включает в себя все подразделения предприятия. Такие системы предоставляют базу данных ресурсов предприятия и полноценные инструменты для планирования, анализа и прогнозирования его ресурсов. Они являются более востребованными в отрасли торговли, машиностроения и строительстве.

Таким образом, ERP-системы обеспечивают производство качественной продукции, максимально эффективно используя имеющиеся ресурсы, а CRM-системы помогают в реализации конкурентоспособной продукции клиентам. Комбинация этих систем позволит малому предприятию выйти на первые места конкурентного рынка за счет грамотного управления ресурсами компании и взаимоотношениями с клиентами.

Бесспорно, на рынке программного обеспечения есть множество ERP, CRM-систем, позволяющих управлять предприятием. Для того, чтобы грамотно выбрать систему, удовлетворяющую требованиям малого предприятия, необходимо понять преимущества и недостатки готовых коммерческих программных продуктов и сделать выводы. Рассмотрим самые популярные[8] и часто используемые программные продукты в своих отраслях, для их более подробного анализа с точки зрения малого предприятия. Для рассмотрения были взяты системы SAP ERP, 1C:ERP и Битрикс24 CRM.

SAP ERP

Преимущества:

- Защита данных с возможностью предоставления доступа определенным пользователям. Это упрощает контроль видимости данных пользователями для руководителя.

- Интеграция всех подразделений предприятия, создание целостной системы. Возможность управления всеми подразделениями увеличит эффективность и позволит анализировать работу каждого из них.

Недостатки:

- Подготовка к внедрению за счет написания точного технического задания, наем консалтинговой фирмы или штатных специалистов

- Высокая цена (от 400 тыс. долларов для малого и среднего бизнеса). Для некоторых предприятий такая цена может составлять существенную часть от доходов.

- Дополнительное обучение для пользования программой. Такое обучение необходимо, так как инструментарий программы не будет привычен обычному пользователю.

- Отсутствие мобильной версии. Это снижает количество различных вариантов использования системы на разных устройствах.

- Интерфейс не дружелюбен для пользователя. Дизайн с вниманием на множество вариантов настроек и контроля, с огромным количеством кнопок и форм могут негативно влиять на понимание программы пользователем.

1C:ERP

Преимущества:

- Готовые конфигурации. Содержит все типовые конфигурации 1С – то есть имеется все, что может понадобиться для работы с системой.

- Производственный учет. В отличие от других конфигураций 1С – 1C:ERP единственная, где реализован полноценный производственный учет.

- Есть мобильное приложение, но каждый сеанс запуска мобильного приложения требует отдельной клиентской лицензии «1С:Предприятие 8»

Недостатки:

- Интерфейс не дружелюбен для пользователя, так как многие документы содержат в себе несколько вкладок с большим количеством различных полей, некоторые пользователи могут не разобраться в такой системе.

- Высокая цена (Самая дешевая версия – 422 тыс. руб., самая дорогая – 2.7 млн. руб.). Хотя цена является одной из самых низких на рынке, особенно если сравнивать с конкурентами, для малого предприятия такое вложение может быть слишком большим, так как составляет значительную часть от его дохода.

- Требуется дополнительное обучение персонала ввиду сложности процесса управления и непривычности обычного пользователя к подобным сложным системам.

- Постоянные обновления, которые могут содержать новые ошибки, могут привести к тому, что будут необходимы изменения в самой конфигурации. Также возникает потребность в тщательном тестировании конфигурации после новых обновлений во избежание ошибок во время эксплуатации программы.

- Слабая производительность из-за того, что при работе с документами многие из них долго загружаются. Данная проблема также присутствует и в других продуктах 1С.

Битрикс24 CRM

Преимущества:

- Есть бесплатная версия системы, но в ней отсутствуют ключевые компоненты коммерческой CRM, такие как: сквозная аналитика, маркетинг, центр продаж. К тому же техническая поддержка предоставляется только при приобретении «Энтерпрайз» редакции системы с минимальной стоимостью в 999 тыс. руб.

- Возможность интеграции с 1С. Большое преимущество для каждой компании, так как интеграция с системой 1С позволяет решить ряд проблем автоматизации бизнес-процессов.

- Простота в управлении клиентским приложением. Интерфейс программы обладает современным дизайном и является достаточно простым.

Недостатки:

- Высокая цена (решение из коробки от 999 тыс. руб., или от 390 тыс. руб. в год)

- Присутствует множество функций, которые могут быть не нужны определенным пользователям, тем самым усложняя и замедляя процесс работы.

Анализ наиболее популярных CRM и ERP-систем показывает несколько общих недостатков этих систем с точки зрения их применения на малых предприятиях: высокая цена самой системы, а также цена её интеграции, отсутствие мобильной версии, либо её присутствие с урезанным функционалом и сложность пользования. Самой минимальной ценой из

рассматриваемых цен стала стоимость 1С:ERP – от 420 тыс. руб., но даже эта цена является сравнительно большой для малого производства, а если учитывать цену интеграции и сопровождения системы, то можно ожидать цену в несколько раз выше заявленной. Ещё один отрицательный момент – отсутствие адаптивности дизайна или мобильной версии приложения. Для многих предприятий, где иногда требуется получить доступ к системе в полевых условиях, обязателен вариант наличия мобильной версии. Возможность адаптации дизайна позволит эффективно управлять необходимыми бизнес-процессами вне зависимости от технических параметров устройства пользователя. Также во всех рассматриваемых системах присутствует проблема со сложностью интерфейса – визуальной составляющей программы. Множество сложных элементов управления и их нагромождение негативно сказываются на восприятии программы и усложняют, замедляют процесс взаимодействия с ней.

1.3. Актуальность применения готовых CRM, ERP систем на малом предприятии

Анализируя полученные результаты сравнения самых популярных и востребованных CRM и ERP-систем на рынке, можно сказать, что интеграция таких систем на малом предприятии имеет существенные сложности и недостатки. Цены данных систем в некоторых случаях могут составлять существенный процент от чистой прибыли предприятия, к тому же маловероятно, что предприятие будет ограничиваться лишь покупкой системы, также необходима ее интеграция в компанию и сопровождение по мере эксплуатации. Отсутствие мобильной версии также является значимым минусом в пользовании системой, так как ограничивает определенные варианты использования программы пользователями. А сложность интерфейса этих систем заставляет обеспечить для сотрудников предприятия дополнительное обучение, для того чтобы у них появились навыки пользования программой.

Эти минусы приводят к следующему заключению – использование готовых коммерческих ERP и CRM-систем на малых предприятиях неактуально и не удовлетворяет многим требованиям предприятий. Поэтому самым эффективным решением для таких предприятий является создание собственной системы управления, которая является комбинацией CRM и ERP-системы, удовлетворяет всем его требованиям и позволит эффективно автоматизировать необходимые бизнес-процессы с максимальной для себя выгодой.

2. Анализ

2.1. Постановка задачи

Необходимо разработать клиентское приложение для системы управления предприятием, которое позволяет создавать новые записи, редактировать и удалять записи типов пользователей с разными уровнями доступа, вести учет рабочего времени сотрудников, трудоемкости деталей и чертежей, списки клиентов, заявок на производство и обратной связи.

Требуется создать API для взаимодействия с данными на сервере и графический интерфейс, с которым будут взаимодействовать пользователи.

Должна быть обеспечена безопасность данных пользователей. Необходимо использовать токен доступа для безопасного обмена данных с сервером.

2.2. Требования к ПО

К разрабатываемой системе управления малым предприятием были выдвинуты следующие требования:

2.2.1. Бизнес-требования

- Система должна позволить эффективно управлять и контролировать заявки на производство от клиентов.
- Грамотное и эффективное взаимодействие с клиентами. Организация действий и задач для выстраивания успешных взаимовыгодных отношений с клиентами.
- Учет рабочего времени должен позволить анализировать рабочее время в каждом из цехов для оптимизации работ в будущем. Также учет рабочего времени должен предоставлять возможность анализа времени работ для каждого вида продукции или чертежа .

2.2.2. Требования безопасности

- Доступ к любым данным должен предоставляться при наличии актуального токена доступа.

2.2.3. Нефункциональные требования

- Приложение должно представлять собой совокупность модулей, которые могут быть быстро модифицированы, заменены или отключены при необходимости.
- Необходимо вести документацию с информацией о всех основных компонентах клиентского приложения.

2.2.4. Качественные характеристики

- При скорости 512 Кбайт/с время загрузки страниц не должно составлять более 2-3 секунд. Система должна использовать минимум ресурсов трафика пользователя и не требовать большие скорости сети Интернет, так как многие сотрудники могут пользоваться ей в полевых условиях по мобильной сети.
- Добиться максимального уровня удобства для пользователей системы:
 - Проверка вводимых пользователем данных перед их сохранением.
 - Страницы для взаимодействия с пользователем должны содержать минимально возможное количество элементов управления, необходимое для выполнения заданных функций.
 - Постоянство дизайна и функционала элементов управления для отсутствия необходимости постоянного обучения пользователя

2.2.5. Требования к интерфейсу

- Минимальный размер экрана - 320 пикселей по ширине, максимальный – 2560 пикселей. Адаптируемая верстка, подходящая для экранов любых размеров, чтобы пользователь смог воспользоваться системой вне зависимости от устройства, которым он пользуется.
- Интерфейс должен быть минималистичен и содержать минимум логических элементов для простоты использования и для легкого понимания пользователем.

2.2.6. Основные функции системы в целом:

- Возможность создавать, редактировать и удалять продукцию и их категории.
- Возможность создавать, редактировать и удалять сотрудников предприятия.
- Возможность создавать, редактировать и удалять клиентов, учет истории работы с ними.
- Возможность создавать, редактировать и удалять заявки на производство продукции в цехах.
- Система должна иметь систему авторизации и защиты данных пользователей.

2.3. Группы пользователей

Приложение должно обеспечить возможность управления пользователями с различными уровнями доступа. Необходимо создать следующие уровни доступа:

- Диспетчер

- Создание, редактирование, удаление записей реестра транспортировок.
- Управление базой сотрудников.
- Управление базой перечня работ.
- Учет собственного рабочего времени и времени сотрудников в офисе (менеджеры, инженеры).
- Инженер
 - Управление базой перечня работ.
 - Учет собственного рабочего времени.
 - Управление базой «Оснастка», в которую входят: база данных «Штампы», база данных «Станки», база данных «Пресс-формы», база данных «Запчасти».
- Цех 1
 - Учет рабочего времени сотрудников.
 - Добавление записей в базу данных «Оснастка», в которую входят: таблица «Штампы», таблица «Станки», таблица «Пресс-формы», таблица «Запчасти».
- Цех 2
 - Добавление записей в базу «Оснастка», в которую входят: база данных «Штампы», база данных «Станки», база данных «Пресс-формы», база данных «Запчасти».
 - Учет рабочего времени.
- Цех 3
 - Добавление записей в базу «Оснастка», в которую входят: база данных «Штампы», база данных «Станки», база данных «Пресс-формы», база данных «Запчасти».
 - Учет рабочего времени.
- Менеджер
 - Создание, редактирование базы клиентов, учет истории работ. Возможность создавать заявки.
 - Учет собственного рабочего времени.
- Руководитель
 - Доступ ко всем разработанным функциям с возможностью редактирования, удаления и создания элементов баз данных.

2.4. Системные требования

- Операционная система: Microsoft Windows XP/Vista/7/8/10, Linux, MacOS.
- Браузер: Internet Explorer 8 и выше, Opera 10.0 и выше, Google Chrome 18.0 и выше, Mozilla Firefox 13.0 и выше, Microsoft Edge 25.0 и выше.
- Скорость сети Интернет: 512 Кбайт/с и выше.

2.5. Сведения о предприятии

Предприятие, для которого разрабатывается данная система, состоит из 49 сотрудников, из них: руководитель, как минимум один менеджер, один диспетчер, как минимум один инженер, начальники цеха (в каждом из цехов) и работники цехов. Всего на данный момент предприятие имеет 3 цеха: Цех 1, Цех 2 и Цех 3, в каждом из которых производится определенный тип продукции и выполняются заказы. Менеджеры взаимодействуют с клиентами, принимают заказы и создают заявки, передаваемые в производственные подразделения. Диспетчер отвечает за реестр транспортировок и список сотрудников. Инженер создает записи о деталях, запчастях, чертежах. Каждый из цехов обновляет статусы заказов на производство, управляет базами склада и заказов. Каждый сотрудник в конце рабочего дня отмечает количество отработанных часов.

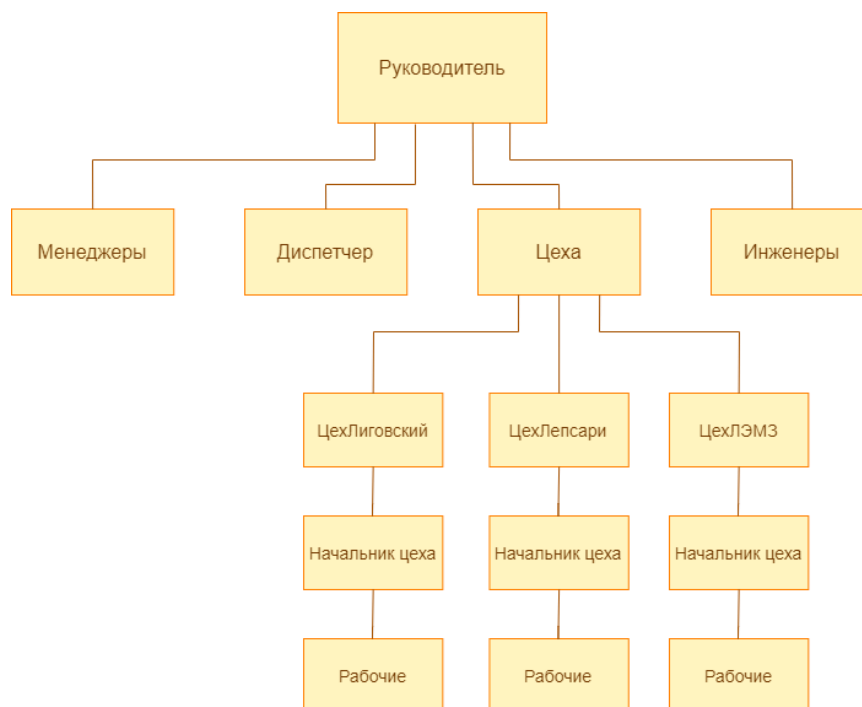


Рис. 1.1. Структура предприятия

Такая структура подразумевает создание нескольких типов пользователей с определенным уровнем доступа и видимостью данных.

Также, так как данное предприятие занимается не только производством, а еще и реализацией своей продукции, необходимо разработать функционал для обеспечения работы менеджера - управление отношений с клиентами, их заказами и заявками.

3. Проектирование

3.1. Компонентная структура ПО

Разрабатываемое программное обеспечение (Рис. 3.1) состоит из нескольких основных элементов: клиентское приложение, серверное приложение, база данных. Предметом моей разработки и этой дипломной работы является клиентское приложение. Клиентское приложение выполняется в браузере, который в свою очередь запускается на устройстве пользователя и взаимодействует с серверным приложением посредством созданного API – программной прослойки для обмена данными между двумя приложениями. Сервер NodeJS, изображенный на Рис. 3.1. запускается на удаленном сервере и необходим для получения пользователем доступа к веб-страницам клиентского приложения.

База данных на PostgreSQL и Сервер на Spring не являются частями разрабатываемого клиентского приложения, и рассматриваются в данной дипломной работе как «черный ящик», так как не являются моей собственной разработкой.

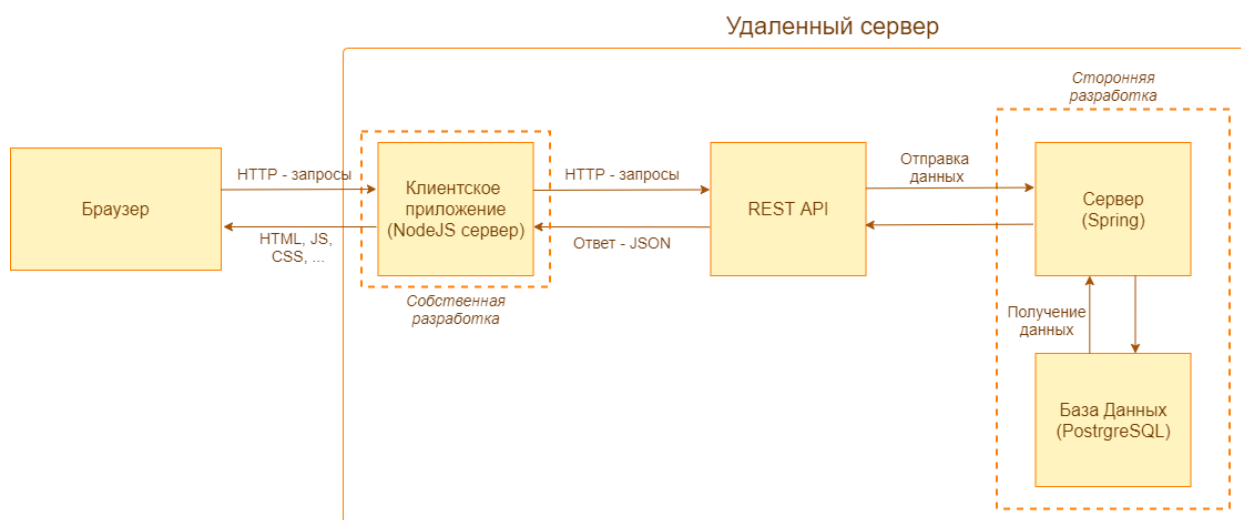


Рис. 3.1. Структура проекта

3.2. Структура данных

Основные модули программы спроектированы по принципу контейнер – компонент. Это один из основных шаблонов проектирования, применяющийся при разработке приложений с использованием библиотеки React. Это значит, что контейнер получает и обрабатывает данные с сервера, которые передаются непосредственно в презентационные компоненты, которые, как правило, не имеют внутреннего состояния. UML диаграммы классов для основных модулей клиентского приложения изображены внизу:

- Обратная связь (Рис. 3.2) – предоставляет возможность создавать обсуждения, изменять их статус и писать сообщения в чате обсуждения. Состоит из форм создания, редактирования обсуждений, страницы просмотра и фильтрации обсуждений по их статусам, чата с сообщениями пользователей с возможностью писать свои сообщения.

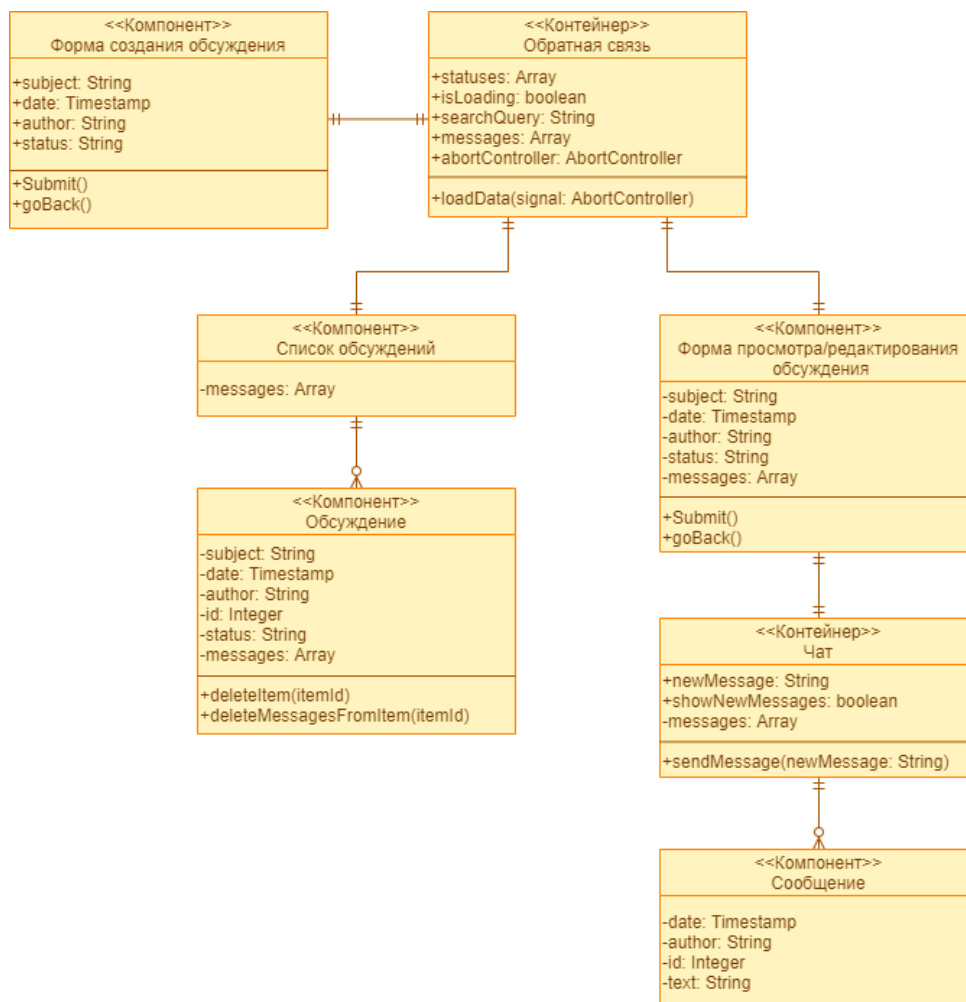


Рис. 3.2. Диаграмма классов для модуля «Обратная связь»

- Клиенты (Рис. 3.3) – Модуль, позволяющий управлять списком клиентов, историей работ с ними и упростить взаимодействие с клиентами. Состоит из форм просмотра, редактирования, создания клиентов, которые включают в себя компоненты выбора юридических лиц, выбора контактных лиц и истории работ. Также в этот модуль входят основная страница просмотра списка клиентов с пагинацией и меню типов клиентов с возможностью сортировки элементов списка клиентов по алфавиту, дате следующего контакта.

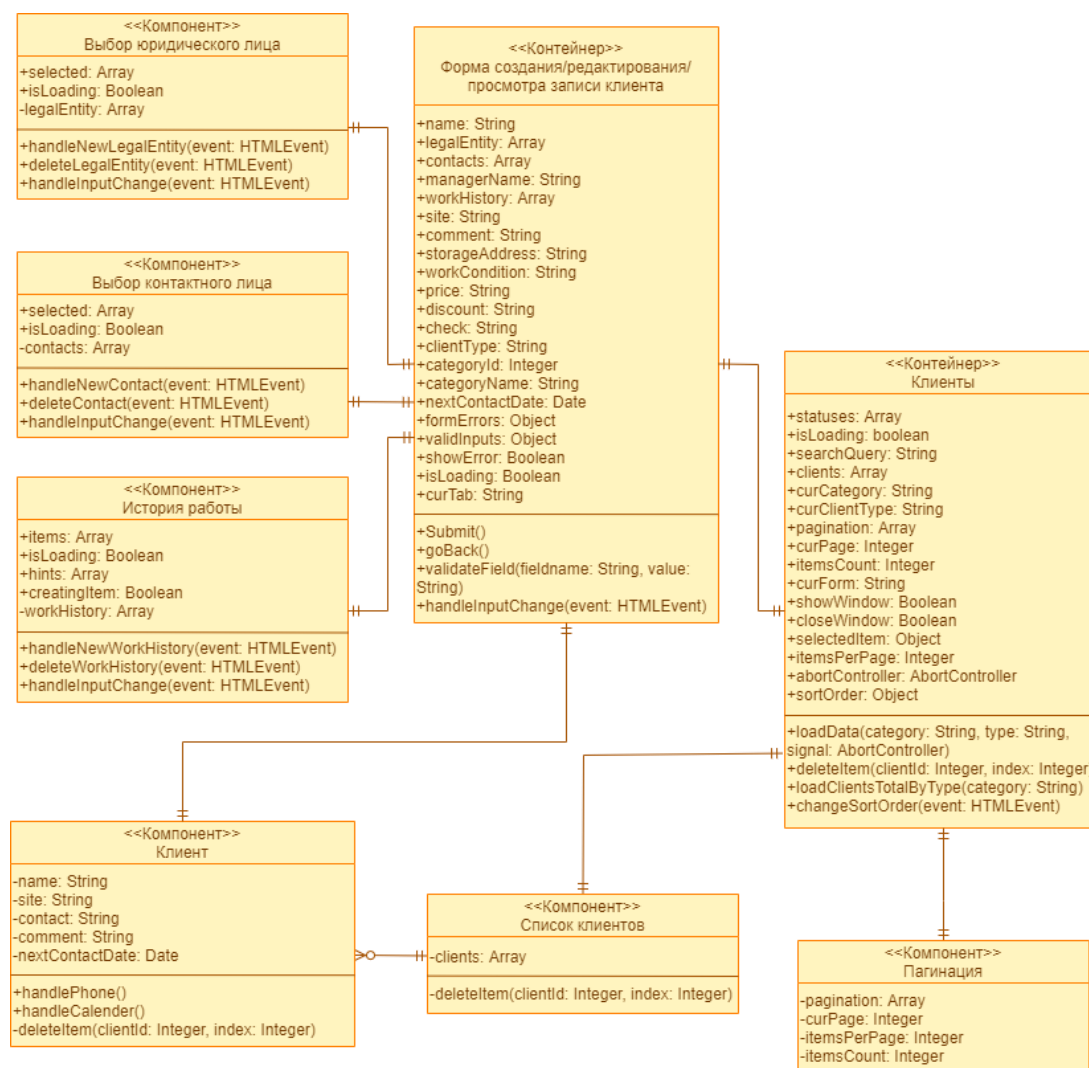


Рис. 3.3. Диаграмма классов для модуля «Клиенты»

- Учет времени сотрудников (Рис. 3.4) – Модуль для контроля времени работы сотрудников предприятия. Состоит из форм создания и редактирования записей о данных учета времени сотрудников, которые содержат компоненты ввода количества часов, выбора продукции/чертежа, выбора сотрудника, выбора типа работы. Основным экраном этого модуля является страница просмотра списка работ за определенный период, с возможностью просмотра каждого вида работы сотрудника за день по часам.

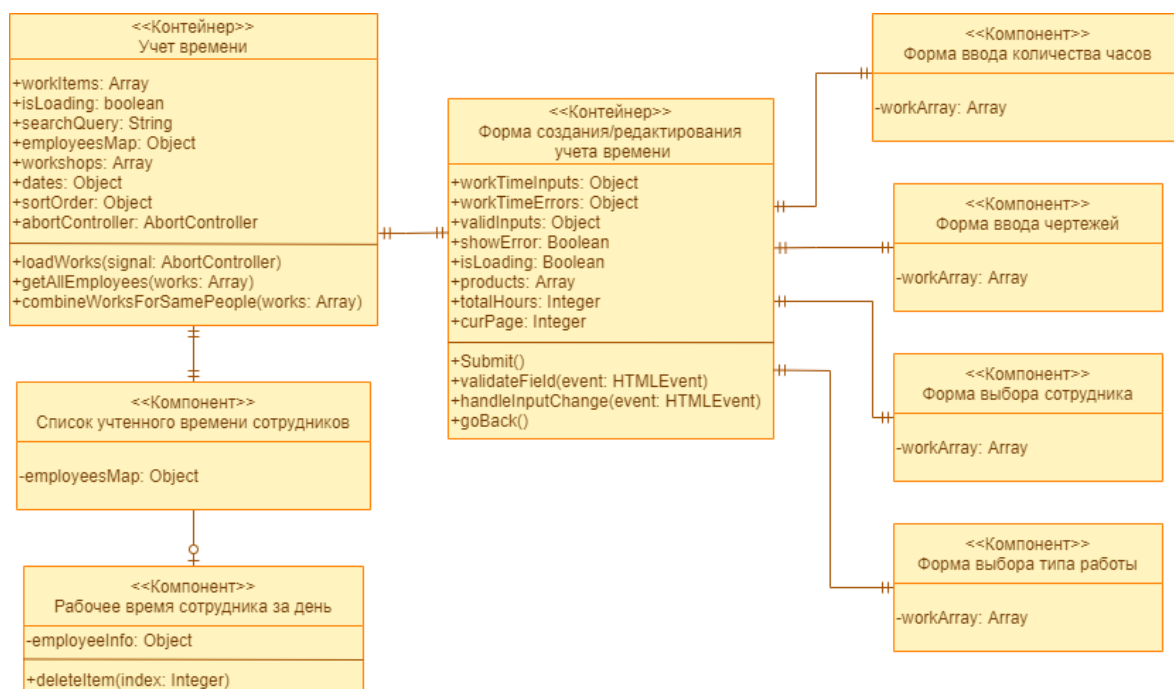


Рис. 3.4. Диаграмма классов для модуля «Учет рабочего времени сотрудников»

3.3. Взаимодействие клиента и сервера

Архитектура клиент-сервер, использованная при разработки данного программного обеспечения, позволяет разделить нагрузку между клиентским приложением и серверным приложением. В данном программном обеспечении используется трехуровневая модель (Рис. 3.5) взаимодействия клиент-сервер.

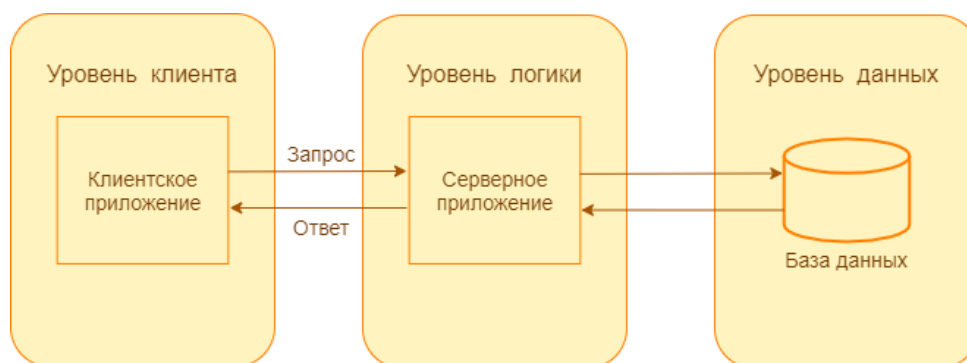


Рис. 3.5. Трехуровневая архитектура модели клиент-сервер

Клиент взаимодействует с серверным приложением непосредственно через API – своего рода прослойкой между двумя программами, для обмена данными между ними. Клиентское приложение посылает HTTP-запросы через вызов функции `fetch()` в API. Типы посылаемых HTTP-запросов могут быть следующими: GET, DELETE, POST, PUT. В ответ сервер посылает код ответа и тело запроса, если такое имеется.

3.4. Структура клиентского приложения

Структура клиентского приложения и составляющих его подсистем и компонентов представлена на Рис. 3.6. Структура разработана таким образом, что наличие каждого модуля не является обязательным для его функционирования, тем самым каждый из модулей можно при необходимости убирать, изменять или добавлять новые.

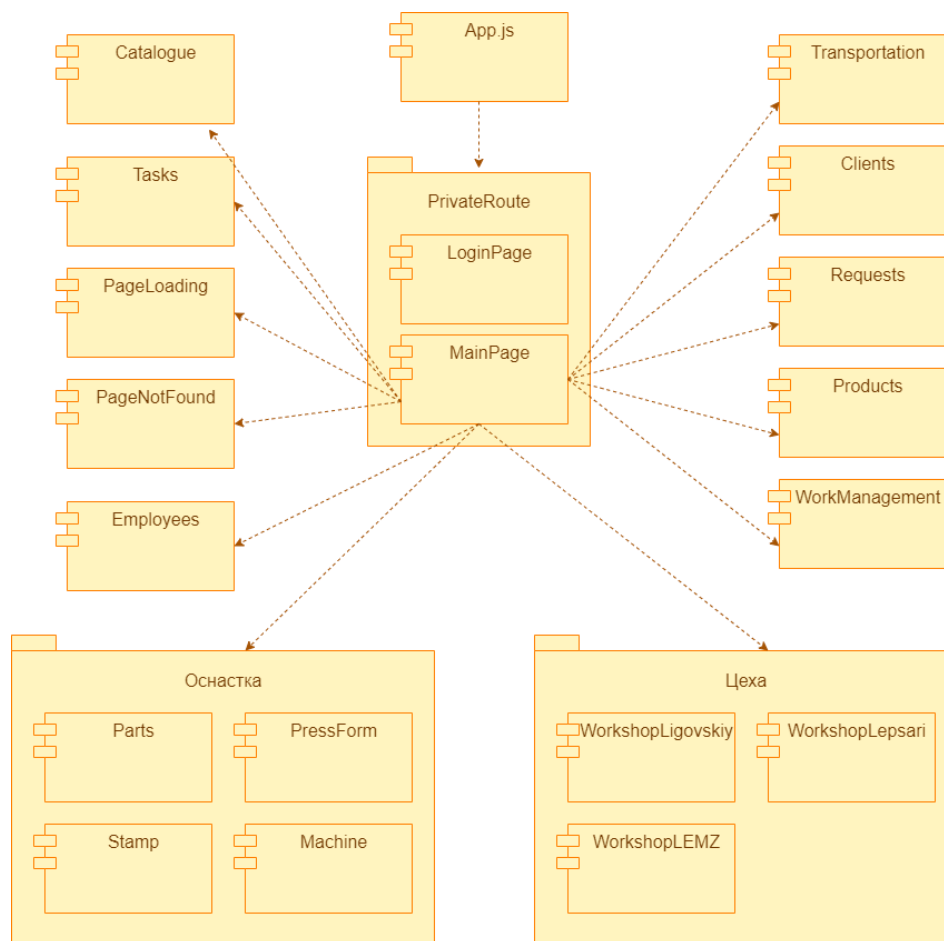


Рис. 3.6. Диаграмма компонентов приложения

Приложение состоит из следующих компонентов:

- App.js – основной компонент системы, который содержит данные о пользователе, авторизован ли он, и определяет нужно ли обновить токен доступа в случае истечения срока его действия.
- PrivateRoute – компонент программы, который определяет, какую страницу отображать для пользователя – страницу авторизации или главную страницу программы, в зависимости от того, авторизован он или нет. Это позволяет предотвратить доступ к программе для неавторизованных пользователей.
- PageNotFound – компонент для отображения сообщения об ошибке о несуществующей странице. Отображается, если путь не совпадает ни с одним путем, соответствующим какому-либо компоненту.

- PageLoading – компонент с индикатором загрузки, отображается при загрузке модулей при переходе пользователя из одного пункта меню в другой.
- LoginPage – страница авторизации пользователя, в случае, если пользователь уже авторизован, предлагает выйти из текущего аккаунта.
- MainPage – это компонент, определяющий, какой из компонентов отображать в зависимости от текущего пути в адресной строке.
- Оснастка – подсистема для управление базами: «Штампы», «Пресс-формы», «Станки», «Запчасти».
- Цеха – подсистема для управления цехами. Каждый цех состоит из компонентов: «Очередь производства», «Склад», «Комплектация цеха».
- Catalogue – компонент для создания и редактирования каталога продукции. Пользователь загружает .xlsx файл со списком цен на продукцию и программа обрабатывает его, создавая .pdf файл с каталогом продукции.
- Tasks – компонент для управления записями о задачах для сотрудников, созданных руководителем. Для каждого пользователя записи фильтруются, чтобы каждый видел только свои записи, где ответственный за их выполнение – сам пользователь.
- Requests – компонент для управления заявками на производство.
- Employees – компонент для управления списком сотрудников.
- Transportation – компонент для управления реестра транспортировок.
- Clients – компонент для управления списком клиентов.
- Products – компонент для управления списком продукции.
- WorkManagement – компонент для создания, редактирования, анализа и просмотра данных по учету рабочего времени сотрудников предприятия.

3.5. Экранные формы

В программе используется множество форм для создания, редактирования, просмотра записей. Для работы с формами и обеспечения возможности использовать один код несколько раз, были специально разработаны следующие компоненты:

- Button – компонент для отображения кнопки. При передаче параметра isLoading(идет ли в данный момент загрузка) текст на кнопке заменяется индикатором загрузки.

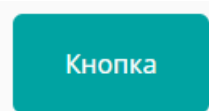


Рис. 3.7. Компонент Button

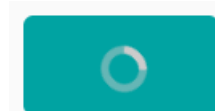


Рис. 3.8. Компонент Button в состоянии загрузки

- **CheckBox** –компонент для отображения полей для отметки – чекбоксов.

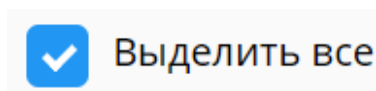


Рис. 2.9. Компонент CheckBox

- **ErrorMessage** – компонент, отображающий всплывающее окно с сообщением об ошибке. Появляется в случае некорректного ввода данных.

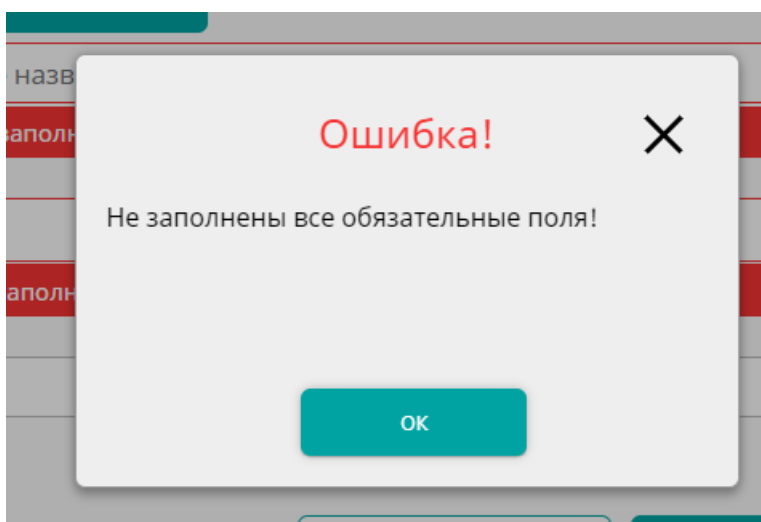


Рис. 3.11. Компонент ErrorMessage

- **FileUploader** – компонент для загрузки пользовательских файлов. Предоставляет возможность ограничить тип данных при загрузке.

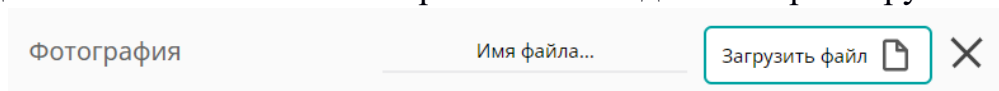


Рис. 3.12. Компонент FileUploader

- **InputText** – компонент текстовой строки.



Рис. 3.13. Компонент InputText

- **InputUser** – компонент, позволяющий выбрать пользователя из списка всех зарегистрированных пользователей. По умолчанию для всех ролей, кроме Руководителя, ставится собственное имя пользователя.

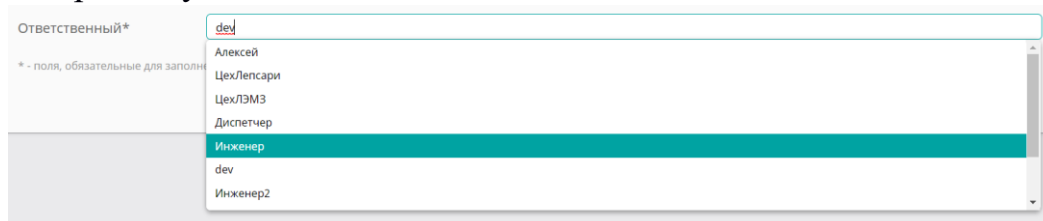


Рис. 3.14. Компонент InputUser

- **InputProducts** – это компонент, который позволяет управлять продукцией для вложенного списка продукции.

Рис. 3.15. Компонент InputProducts

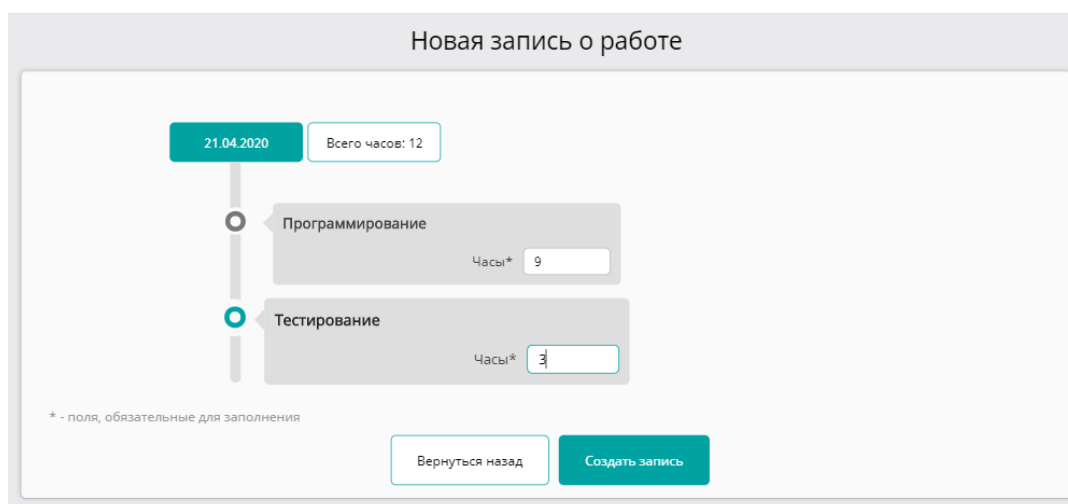
- SearchBar – это компонент для ввода пользователем поискового запроса, с помощью которого выполняется фильтрации данных.

Рис. 3.16. Компонент SearchBar

Основные экранные формы включают в себя:

- Учет рабочего времени (Рис. 3.17)
Включает в себя выбор даты, выбор сотрудника из базы «Сотрудники», список работ, выполнявшихся сотрудником, и форму для ввода количества часов, отработанных сотрудником для каждого типа работ.

Рис.3.17. Интерфейс учета рабочего времени



Новая запись о работе

21.04.2020 Всего часов: 12

Программирование Часы* 9

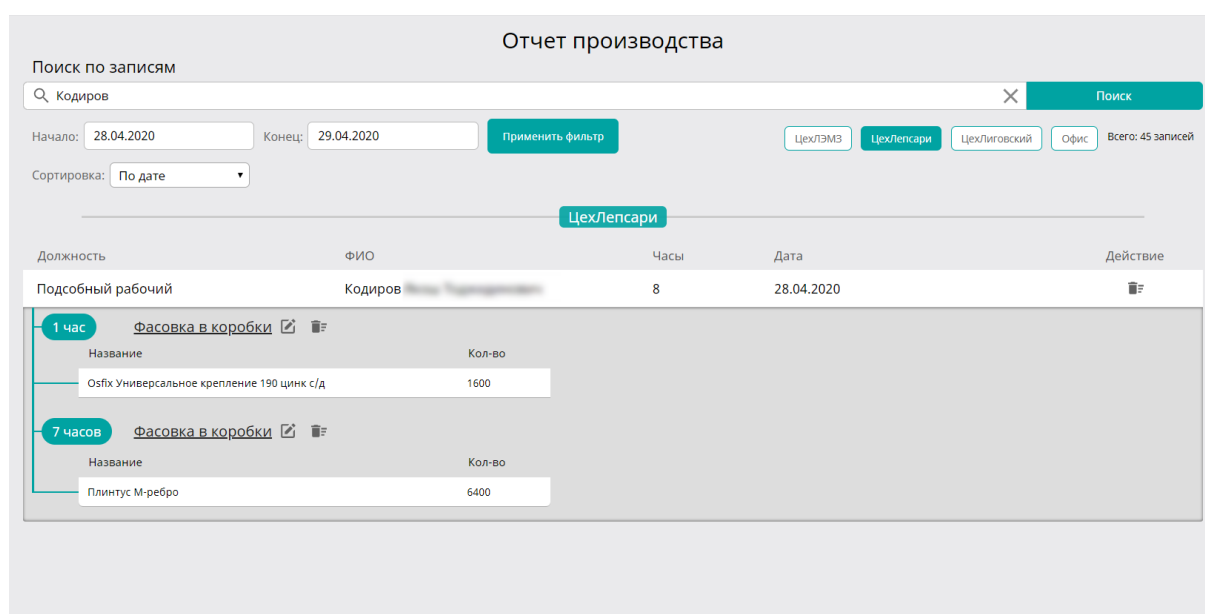
Тестирование Часы* 3

* - поля, обязательные для заполнения

Вернуться назад Создать запись

Рис. 3.18. Интерфейс учета рабочего времени

Окно отчета производства (Рис. 3.19) позволяет просмотреть записи сотрудников об их учтенном времени в указанный пользователем период.



Отчет производства

Поиск по записям

Кодиров

Начало: 28.04.2020 Конец: 29.04.2020 Применить фильтр

Сортировка: По дате

ЦехЛЭМЗ ЦехЛепсари ЦехЛиговский Офис Всего: 45 записей

Должность	ФИО	Часы	Дата	Действие
Подсобный рабочий	Кодиров	8	28.04.2020	
1 час Фасовка в коробки				
Название Кол-во				
Осfix Универсальное крепление 190 цинк с/д 1600				
7 часов Фасовка в коробки				
Название Кол-во				
Плентус М-ребро 6400				

Рис. 3.19. Окно отчета производства

- Клиенты (Рис. 3.20)

Данное окно предоставляет возможность перехода между тремя типами клиентов: Активные, Потенциальные, В разработке. Компонент поиска предоставляет возможность поиска по названию, комментарию, дате следующего контакта по всем записям клиентов. Основную часть страницы составляет таблица с данными, которые можно отсортировать по алфавиту и дате следующего контакта. Система вывода данных по страницам позволяет уменьшить время загрузки и облегчить процесс работы с программой.

Поиск по клиентам						
Введите запрос для поиска...						Поиск
Активные (11) Потенциальные (650) В разработке (53)						
Сортировка: По алфавиту (A-Z)						
Всего: 650 записей						
Название	Сайт	Контакты	Комментарий	Дата след. контакта	Действие	
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	24.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	25.11.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	22.11.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	14.05.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	26.03.2020	☆	☎
ИП	www.123.ru	8800-123-4567	Иванов Иван Иванович	30.03.2020	☆	☎
Кол-во элем. на странице: 10						
Пред 1 2 3 4 5 ... 65 След						

Рис. 3.20. Страница просмотра клиентов

- Заявки (Рис. 3.21)

Форма создания заявки включает в себя ввод даты создания, списка продукции, которую необходимо произвести, кодовое слово заказчика и ответственного за выполнение заказа. Основной страницей

Новая заявка

Дата*

29.04.2020

Добавить продукцию

Введите название продукта для поиска...

Выбранная продукция:

Продукция1

Кол-во* 10000

Фасовка* Коробка

Кодовое слово*

ООО "Компания"

Ответственный*

test

* - поля, обязательные для заполнения

Вернуться назад

Оформить заявку

Рис. 3.21. Окно создания заявки

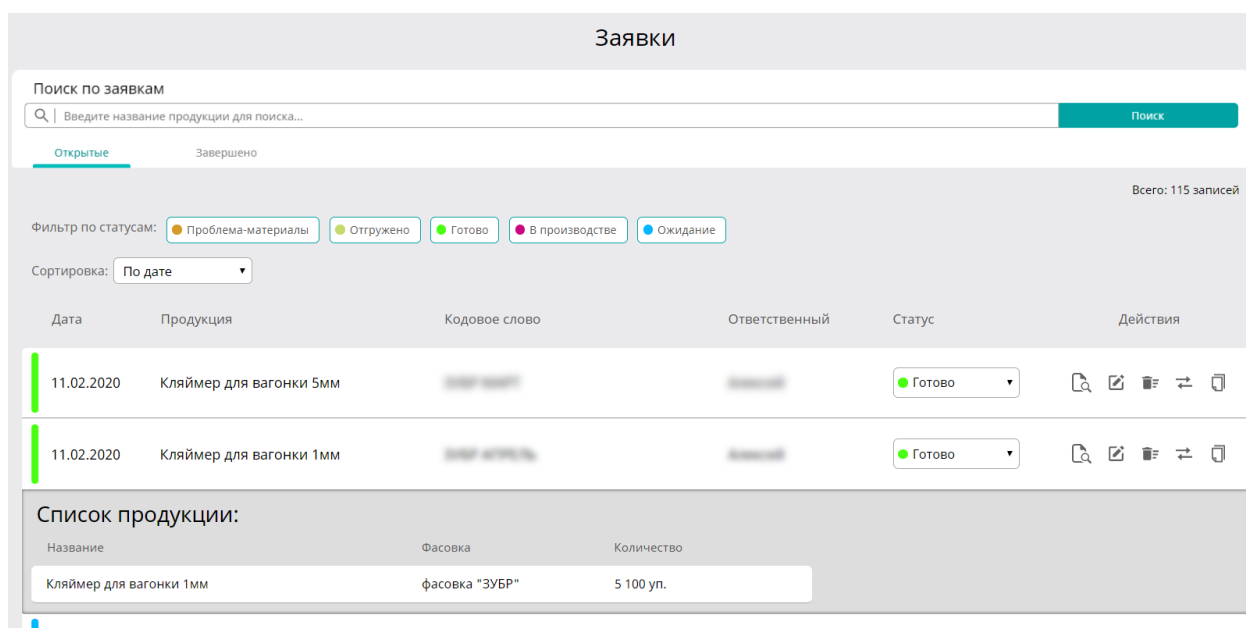


Рис. 4.22. Страница для управления заявками

Кроме перечисленных выше форм, также в программе присутствуют:

- Формы обратной связи.
- Формы заявок на производство в Цехе 2 и Цехе 1.
- Формы продукции.
- Формы реестра транспортировок.
- Формы сотрудников .
- Формы оснасток: формы таблиц «Штампы», «Станки», «Пресс-формы», «Запчасти».
- Формы основных задач.
- Формы списка работ.

4. Разработка приложения

4.1. Методология разработки

В качестве методологии разработки была выбрана концепция Agile – методология быстрой, итеративной разработки приложений. Данная методика позволяет проводить разработку итерациями, которые длятся приблизительно две недели, после каждой итерации заказчику предоставляется готовый продукт.

Так как для разрабатываемого программного продукта отсутствует точное и конечное техническое задание, требования могут быстро меняться, сроки запуска проекта сжаты, и ключевым является общение с заказчиком лично - выбор пал именно на эту методику. Также рассматривались другие популярные методики как Rapid Application Development (RAD), методология «Водопад», которые не подходили по ряду причин:

RAD:

- Требуется создание прототипа.
- Наличие руководителя разработки.

«Водопад»

- Подходит для больших проектов.
- Требуется четкое и точное техническое задание, так как при появлении новых требований или задач, нельзя вернуться на этап проектирования – разработка ведется поэтапно, последовательно
- Наличие руководителя разработки.

Следовательно было принято решение выбрать методологию Agile для разработки.

4.2. Инструменты разработки

Разрабатывать клиентское приложение было решено на языках JavaScript, CSS, HTML. Данные технологии позволяют разработать клиентское приложение доступное на всех современных видах устройств и большинстве браузеров.

4.2.1. Языки программирования

- JavaScript – язык для написания скриптов, описывающих поведение веб-страницы. С помощью него производится загрузка данных, их обработка, выполнение множества функций, необходимых для корректного функционирования клиентского приложения
- CSS – язык для описания внешнего вида документа. С его помощью описываются стили для элементов веб-страниц, включая как цвет текста, так и анимация определенных элементов.
- HTML – язык разметки веб-документов. Содержит основную структуру веб-страницы. Также к нему подключаются все написанные модули как

модули стилей, написанные на CSS, так и модули поведения, написанные на JavaScript.

4.2.2. Библиотеки и фреймворки

- React – библиотека JavaScript, позволяющая ускорить и упростить разработку приложений. Имеет виртуальный DOM, и перезагружая только компоненты, в которых произошли изменения, ускоряет время перезагрузки страницы. При разработке приложений на React используется идея создания компонентов – модульных элементов, которые можно использовать несколько раз в различных частях программы.
- SASS – препроцессор CSS. Предоставляет дополнительные возможности работы с CSS, такие как: вложенные правила, примеси (шаблон для указания конкретного поведения). При разработке используется вариант синтаксиса SASS – SCSS.
- NodeJS – платформа для создания серверного приложения. Используется для получения пользователем доступа к клиентскому приложению на выделенном сервере. Необходим только для того чтобы пользователь получил доступ к разработанным веб-страницам клиентского приложения.

4.2.3. Программы и сервисы

- GitHub – сервис для управления версиями проектов и ведения нескольких веток разработки. При разработке используются две ветки проекта:
 - master - основная ветка, файлы которой запущены на удаленном сервере. Обновляется при условии прохождения тестирования обновлений из ветки develop.
 - develop – ветка активной разработки, которая обновляется каждый раз при введении новых функций или исправлений ошибок.
- Visual Studio Code – редактор исходного кода. Поддерживает разработку веб-приложений, Git, и других популярных средств разработки. Содержит огромную библиотеку пользовательских расширений, предоставляющий множество дополнительных функций для обеспечения более удобной разработки.
- PostgreSQL Admin – объектно-реляционная система управления базами данных. Используется для просмотра данных, хранящихся в базе данных.
- IntelliJ Idea – среда разработки программного обеспечения. Используется для работы с серверным приложением написанным на Java.

4.3. Разработка классов

Были разработаны классы для всех модулей, упомянутых в разделе Проектирование. Ниже представлены методы и свойства классов для следующих основных модулей:

- FeedbackPage, в который входят классы:
 - NewFeedback – класс, описывающий форму для создания обсуждения.
 - ViewFeedback – класс, описывающий форму для просмотра обсуждения и сообщений в чате.
 - FeedbackChat – класс, представляющий собой модуль для отображения чата.
 - FeedbackPage – класс, описывающий главную страницу просмотра списка данных обратной связи.
- WorkManagement:
 - NewRecordWork – класс, описывающий форму для создания записи о данных отработанных сотрудником часов.
 - EditRecordWork – класс, описывающий форму для редактирования записи о данных отработанных сотрудником часов.
 - SelectWorkHours – класс, представляющий собой модуль записи количества часов для определенного списка работ совершенных сотрудником.
 - WorkManagementPage – класс, предназначенный для отображения списка учтенного времени работы сотрудников за указанный пользователем период.
- Clients:
 - ClientForm – класс, описывающий форму для создания, редактирования, просмотра данных о клиенте.
 - ClientsPage – класс, описывающий главную страницу просмотра данных о клиентах определенных категории и типа.
 - SelectItems – класс, представляющий собой форму для управления данными о вложенных элементах (Контактные лица, Юридические лица, История работы) клиента.
- Requests:
 - RequestForm – класс, описывающий форму для создание, редактирования, просмотра данных о заявке.
 - RequestsPage – класс, описывающий главную страницу просмотра данных о заявках.

4.3.1. FeedbackPage – Обратная связь

Класс NewFeedback/ViewFeedback

`handleInputChange(event)` – метод для установления нового состояния при изменении данных, также проверяет корректность введенных данных и при необходимости хранит сообщение об ошибке в состоянии.

`validateField(fieldName, value)` – метод для проверка корректности введенных данных в зависимости от имени поля, в котором были введены данные.

`formIsValid()` – метод, позволяющий определить есть ли ошибки при заполнении пользователем формы, проходя по всем полям, которые необходимо ввести. В случае наличия ошибок – обновляет переменную состояния `showError` в положение `true`.

`handleSubmit()` – метод, который сначала проверяет корректность введенных данных, а затем отправляет данные на сервер в случае отсутствия ошибок.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`formInputs` – свойство класса, хранящее в себе значения введенных пользователем данных.

`formErrors` – свойство класса, которое хранит данные типа `Boolean`, которые определяют наличие ошибок в обязательных для ввода полях пользователя.

`showError` – свойство класса типа `Boolean`, определяющее необходимость отображения сообщения об ошибке при некорректном вводе данных в форме.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Добавить обсуждение». Значение по-умолчанию – `false`.

Класс FeedbackChat

`newMessage` – свойство класса, хранящее новое сообщение, которое вводит пользователь. При его отправке – поле очищается и становится равным пустой строке.

`showNewMessages` – свойство класса типа `Boolean`, отвечающее за отображение индикатора новых сообщений при наличии соответствующих сообщений.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

Класс FeedbackPage

`getFeedback()` – функция для получения списка данных обратной связи от сервера. Возвращает массив данных типа `Object`.

`deleteFeedbackById(id)` – функция для отправки запроса на удаления записи данных обратной связи по номеру элемента(`id`) в базе данных.

`getMessagesByDiscussionId(id)` – функция для получения списка сообщений в обсуждении по номеру обсуждения(`id`). Возвращает массив данных типа `Object`.

`deleteMessage(id)` – функция для отправки запроса на удаления записи сообщения по номеру этого элемента(`id`) в базе данных.

`formatDateString(Date)` – функция для форматирования данных типа `Date`, в данные формата “дд.мм.гггг”, возвращает данные типа `String` с вышеупомянутым форматом.

`formatDateStringWithTime(Date)` – функция для форматирования данных (`Date`) типа `Date`, в данные формата “дд.мм.гггг чч:мм”, возвращает данные типа `String` с вышеупомянутым форматом.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`loadData(signal)` – метод для отправки запроса на получение списка данных об обсуждениях и сохранения полученных данных в свойстве класса – `messages`.

`searchQuery` – свойство класса, хранящее данные типа `String` со значением текущего поискового запроса. Используется при фильтрации данных об обсуждениях.

`messages` – свойство класса типа `Array`, хранящее массив данных о сообщениях в обсуждении. Используется при просмотре обсуждения.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки любых данных на текущей странице. Значение по умолчанию – `false`.

`statuses` – свойство класса типа `Array`. Содержит массив объектов-статусах обсуждений.

`abortController` – свойство класса тип `AbortController`. Необходимо для предотвращения лишней загрузки данных при выходе со страницы обратной связи.

Следующий код демонстрирует функцию `useEffect` и определение метода `loadData`:

```
useEffect(() => {  
  let abortController = new AbortController()  
  loadData(abortController.signal)  
  return function cancel() {  
    abortController.abort()  
  }  
}, [])
```

```
const loadData = (signal) => {
  getFeedback(signal)
    .then((res) => res.json())
    .then((res) => {
      setMessages(res)
      setIsLoading(false)
    })
}
```

Следующий код демонстрирует элемент, при нажатии на который удаляется выбранное обсуждение:

```
{props.userHasAccess(['ROLE_ADMIN']) && (
  <div
    className="main-window__action"
    title="Удаление чата"
    onClick={(event) => {
      event.preventDefault()
      getMessagesByDiscussionId(item.id)
        .then((res) => res.json())
        .then((res) => {
          return Promise.all(
            res.map((message) => {
              return deleteMessage(message.id)
            })
          )
        })
        .then(() => {
          return deleteFeedbackById(item.id)
        })
        .then(() => {
          return loadData()
        })
      })
  >
  <img className="main-window__img" src={deleteSVG} />
</div>
)}
```

4.3.2. WorkManagement – Учет рабочего времени

NewRecordWork

`handleInputChange(event)` – метод для установления нового состояния при изменении данных, также проверяет корректность введенных данных и при необходимости хранит сообщение об ошибке в состоянии.

`validateField(fieldName, value)` – метод для проверка корректности введенных данных в зависимости от имени поля, в котором были введены данные.

`formIsValid()` – метод, позволяющий определить есть ли ошибки при заполнении пользователем формы, проходя по всем полям, которые

необходимо ввести. В случае наличия ошибок – обновляет переменную состояния `showError` в положение `true`.

`handleSubmit()` – метод класса для обработки данных отработанных часов сотрудника, введенных пользователем и отправке их на сервер.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Создать запись». Значение по-умолчанию – `false`.

`workTimeInputs` – свойство класса, хранящее в себе значения введенных пользователем данных.

`workTimeErrors` – свойство класса, которое хранит данные типа `Boolean`, которые определяют наличие ошибок в обязательных для ввода полях пользователя.

`showError` – свойство класса типа `Boolean`, определяющее необходимость отображения сообщения об ошибке при некорректном вводе данных в форме.

`totalHours` – свойство класса, содержащее общее количество учтенных часов, учитывая все виды введенных пользователями работ.

`currentPage` – свойство класса, хранящее название текущей страницы с формой: 0 – форма создания, 1 – форма ввода количества часов.

`wrapperHeight` – свойство класса для хранения высоты элемента, содержащего форму создания и форму заполнения количества часов для динамического обновления при изменении данных в формах.

WorkManagementPage

`loadWorks(signal)` – метод для отправки запроса на получение списка данных о списке работ и отработанных часов сотрудников и обработка этих данных для суммирования нескольких работ сотрудников, проведенных в один день.

`changeSortOrder(event)` – метод для изменения текущего поля для сортировки данных в таблице.

`getAllEmployees(works)` – метод для обработки данных о работах типа `Array – works` и получения списка всех сотрудников, возвращая данные типа `Object`.

`combineWorksForSamePeople(works)` – метод для обработки данных о работах типа `Array – works` и получения списка всех работ проведенных каждым из сотрудников в заданные дни.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных. Значение по-умолчанию – `false`.

`workshops` – свойство класса типа `Array`, хранящее объекты с информацией о всех подразделениях, выводимых на странице, а также уровни доступа к ним для определенных пользователей.

`searchQuery` – свойство класса, хранящее данные типа `String` со значением текущего поискового запроса. Используется при фильтрации данных об обсуждениях.

`employeesMap` – свойство класса типа `Object`. Представляет собой список всех работ проведенных каждым из сотрудников с разделением по конкретным дням.

`dates` – свойство класса типа `Object`. Хранит дату начала и конца периода для фильтрации данных. Значения по умолчанию: начальная дата – вчерашний день, конечная дата – сегодняшний день.

`sortOrder` – свойство класса, хранящее текущее поле для сортировки данных на странице.

Следующий код демонстрирует элемент фильтрации данных отработанных часов по подразделениям с выводом количества элементов в списке:

```
<div className="work-management-page__workshop-pick">
  {workshops.map((item, index) => {
    if (props.userHasAccess(item.visibility)) {
      return (
        <div
          className={
            item.active
              ? 'main-window__button'
              : 'main-window__button main-window__button--inverted'
          }
          onClick={() => {
            let temp = workshops
            temp.splice(index, 1, {
              ...temp[index],
              name: item.name,
              active: !item.active,
            })
            setWorkshops([...temp])
          }}
        >
          {item.name}
        </div>
      )
    }
  })}
  <div className="main-window__amount_table">
    Всего: {workItems.length} записей
  </div>
</div>
```

4.3.3. Clients – Клиенты

ClientForm

`validateField(fieldName, value)` – метод для проверки корректности введенных данных в зависимости от имени поля, в котором были введены данные.

`formIsValid()` – метод, позволяющий определить есть ли ошибки при заполнении пользователем формы, проходя по всем полям, которые необходимо ввести. В случае наличия ошибок – обновляет переменную состояния `showError` в положение `true`.

`handleSubmit()` – метод класса для обработки данных отработанных часов сотрудника, введенных пользователем и отправке их на сервер.

`handleInputChange(event)` – метод для установления нового состояния при изменении данных, также проверяет корректность введенных данных и при необходимости хранит сообщение об ошибке в состоянии.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`clientInputs` – свойство класса, хранящее все данные клиента, и изменяющееся по мере заполнения пользователем формы.

`formErrors` – свойство класса, которое хранит данные типа `Boolean`, которые определяют наличие ошибок в обязательных для ввода полях пользователя.

`showError` – свойство класса типа `Boolean`, определяющее необходимость отображения сообщения об ошибке при некорректном вводе данных в форме.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Создать запись». Значение по-умолчанию – `false`.

`users` – свойство класса хранящее роли пользователей для ввода в поле «Ответственный менеджер», получающее данные в зависимости от текущей роли пользователя.

`curTab` – свойство класса для хранения информации о текущей странице в форме: `clientData` (информация о клиенте) или `workHistory` (история работ с клиентом). Значение по умолчанию – `clientData`.

ClientsPage

`deleteItem(clientId, index)` – метод класса для удаления записи о клиенте, включая записи о его контактных и юридических лиц.

`loadClientsTotalByType(category)` – метода класса для получения общего количества записей о клиентах для всех типов клиентов (Активные, Потенциальные, В разработке) в текущей категории.

`loadData(category, type, signal)` – метод класса для получение данных о клиентах от сервера, получающий параметры текущей категории и типа

клиентов, а также `signal` для отмены загрузки данных в случае перехода пользователя на другие страницы.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`clients` – свойство класса типа `Array`, хранящее список данных о клиентах.

`curCategory` – свойство класса типа `String`, хранящее текущую категорию клиента.

`curClientType` – свойство класса типа `String`, хранящее текущий тип клиента.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Создать запись». Значение по-умолчанию – `false`.

`searchQuery` – свойство класса, хранящее данные типа `String` со значением текущего поискового запроса. Используется при фильтрации данных о клиентах.

`pagination` – свойство класса типа `Array`, содержащее информацию о страницах для постраничного вывода данных о клиентах.

`curPage` – свойство класса типа `Integer`, определяющее номер текущей страницы для вывода данных о клиентах.

`itemsCount` – свойство класса типа `Integer` для хранения общего количества записей в базе данных.

`curForm` – свойство класса для хранения информации о текущей форме, которая отображается в отдельном окне. По умолчанию – `nextContactDate` (форма редактирования даты следующего контакта).

`showWindow` – свойство класса типа `Boolean`, предназначенное для отображения или скрытия окна с определенными формами.

`itemsPerPage` – свойство класса типа `Integer` для хранения количества записей для одновременного вывода на одной странице.

`sortOrder` – свойство класса, хранящее текущее поле для сортировки данных на странице.

SelectItems

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента, проверяющий нет ли переданные в него параметров по умолчанию.

`clickOnForm(event)` – метод, выполняющийся при нажатии на шапку панели выбора элементов для сворачивания в минимизированное состояние или возврат в нормальное состояние.

`handleNewItem(event)` – метод, добавляющий новый объект элемента в свойство `selected`. Вызывается при нажатии на кнопку «Добавить элемент»

`deleteItem(event)` – метод для удаления записи об элементе.

handleInputChange(event) – метод для установления нового состояния при изменении данных, также проверяет корректность введенных данных.

selected – свойство класса для хранения выбранных элементов.

Следующий код демонстрирует проверку формы на корректность заполнения и отправка запросов на сохранение данных на сервере:

```
formIsValid() &&
  addClient({
    clientType: clientInputs.clientType,
    comment: clientInputs.comment,
    manager: clientInputs.managerName,
    name: clientInputs.name,
    price: clientInputs.price,
    site: clientInputs.site,
    storageAddress: clientInputs.storageAddress,
    workCondition: clientInputs.workCondition,
    check: clientInputs.check,
    nextDateContact: clientInputs.nextContactDate.getTime() / 1000,
    categoryId: clientInputs.categoryId,
  })
  .then((res) => res.json())
  .then((res) => {
    clientId = res.id
    setIsLoading(false)
    return Promise.all(
      clientInputs.legalEntity.map((item) => {
        return addClientLegalEntity({
          name: item.name,
          inn: item.inn,
          kpp: item.kpp,
          ogrn: item.ogrn,
          bik: item.bik,
          checkingAccount: item.checkingAccount,
          legalAddress: item.legalAddress,
          factualAddress: item.factualAddress,
          legalEntity: item.legalEntity,
          clientId: res.id,
        })
      })
    ),
  )
  .catch((error) => {
    setIsLoading(false)
    alert('Ошибка при добавлении записи')
    console.log(error)
  })
```

4.3.4. Requests – Заявки на производство

RequestForm

`validateField(fieldName, value)` – метод для проверки корректности введенных данных в зависимости от имени поля, в котором были введены данные.

`formIsValid()` – метод, позволяющий определить есть ли ошибки при заполнении пользователем формы, проходя по всем полям, которые необходимо ввести. В случае наличия ошибок – обновляет переменную состояния `showError` в положение `true`.

`handleSubmit()` – метод класса для обработки данных о заявке, введенных пользователем и отправке их на сервер.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`handleInputChange(event)` – метод для установления нового состояния при изменении данных, также проверяет корректность введенных данных и при необходимости хранит сообщение об ошибке в состоянии.

`handleDateChange(event)` – метод для установления нового состояния поля даты создания заявки.

`handleProductsChange(event)` – метод для установления нового состояния поля вложенной продукции в заявке.

`handleResponsibleChange(event)` – метод для установления нового состояния поля ответственного текущей заявки.

`requestInputs` – свойство класса, хранящее в себе значения введенных пользователем данных о заявке.

`formErrors` – свойство класса, которое хранит данные типа `Boolean`, которые определяют наличие ошибок в обязательных для ввода полях пользователя.

`showError` – свойство класса типа `Boolean`, определяющее необходимость отображения сообщения об ошибке при некорректном вводе данных в форме.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Добавить запись». Значение по-умолчанию – `false`.

RequestsPage

`deleteItem(event)` – метод класса для удаления данных о заявке.

`getClientsFromRequests (requests)` – метод для получения списка общих клиентов в заявках для возможности последующей фильтрации данных о заявках.

`loadRequests(signal)` – метод для получения данных о заявках с сервера.

`transferRequests(id)` – метод для переноса заявки по номеру `id` в одно из подразделений.

`copyRequest(id)` – метод для копирования данных о заявке с номером `id`.

`useEffect()` – метод, выполняющийся при первом рендере страницы, а также при последующих изменениях одного из состояний компонента.

`requests` – свойство класса `Array` содержащее список данных о заявках.

`searchQuery` – свойство класса, хранящее данные типа `String` со значением текущего поискового запроса. Используется при фильтрации данных о заявках.

`showWindow` – свойство класса типа `Boolean`, предназначенное для отображения или скрытия окна с определенными формами.

`isLoading` – свойство класса типа `Boolean` для хранения данных о текущем состоянии загрузки данных при нажатии на кнопку «Добавить запись». Значение по-умолчанию – `false`.

`toWorkshop` – свойство класса типа `String` для хранения данных о подразделении в которые будет произведен возможный перенос данных о заявке.

`requestId` – свойство класса типа `Integer`, отвечающее за хранение номера `id` заявки.

`clients` – свойство класса типа `Array`, хранящее данные о клиентах, полученных с помощью метода `getClientsFromRequests(requests)`.

`curPage` – свойство класса типа `String`, содержащее информацию о текущей странице просмотра данных о заявках. Значение по умолчанию – вкладка «В процессе».

`requestStatuses` – свойство класса типа `Array`, хранящее данные о статусах заявок с их уровнями доступа и их кодовыми именами.

Следующий код демонстрирует метод фильтрации данных с помощью поискового запроса `SearchQuery(data)` и метод для изменения статуса заявки `handleStatusChange(event)`:

```
const searchQuery = (data) => {
  const query = props.searchQuery.toLowerCase()
  return data.filter((item) => {
    return item.requestProducts.length !== 0 &&
      item.requestProducts[0].name !== null
      ? item.requestProducts[0].name.toLowerCase().includes(query) ||
        item.id.toString().includes(query) ||
        formatDateString(item.date).includes(query) ||
        item.codeWord.toLowerCase().includes(query) ||
        item.status.toLowerCase().includes(query) ||
        item.responsible.toLowerCase().includes(query) ||
        formatDateString(item.shippingDate).includes(query)
      : item.status.toLowerCase().includes(query)
  })
}

const handleStatusChange = (event) => {
  const status = event.target.value
```

```
const id = event.target.getAttribute('id')
return editRequestStatus(
  {
    status: status,
  },
  id,
)
.then(() => {
  props.loadData()
})
.catch((error) => {
  console.log(error)
})
}
```

5. Краткое руководство пользователя

Данное краткое руководство предназначено для ознакомления пользователя с процессом авторизации на сайте, основными элементами управления данными и с процессом взаимодействия с клиентским приложением.

В первую очередь для того, чтобы войти в программу, пользователю требуется иметь логин и пароль от своей учетной записи, которую создает непосредственно руководитель. Вход на сайт расположен по пути /login, при переходе на который пользователю требуется ввести свои данные для входа.

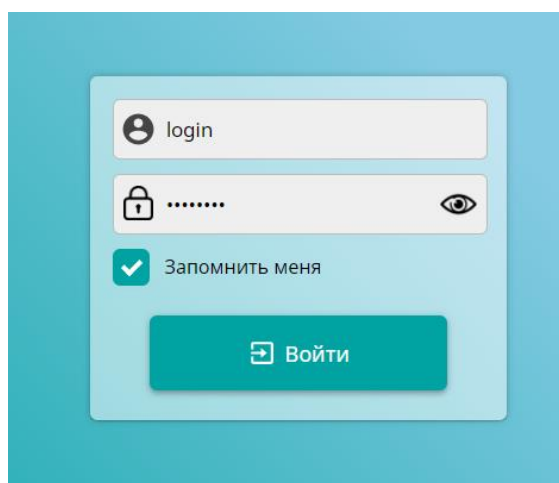


Рис. 5.1. Форма авторизации на сайте

При успешном входе в свою учетную запись пользователь попадет на главную страницу. Для перехода на различные страницы клиентского приложения существует несколько основных элементов – это панель бокового меню (Рис. 5.2) и Шапка сайта (Рис. 5.3). Боковое меню позволяет переходить на все имеющиеся страницы сайта, а также наличие кнопок «плюс» позволяет быстро переходить на страницу создания данных соответствующих элементов. Шапка сайта включает кнопку открытия или закрытия бокового меню, данные текущего аккаунта пользователя и кнопку выхода из системы. Панель с текущими данными пользователя, при нажатии, открывает всплывающее поле с возможностью перехода на определенные страницы, связанные с данными пользователя.

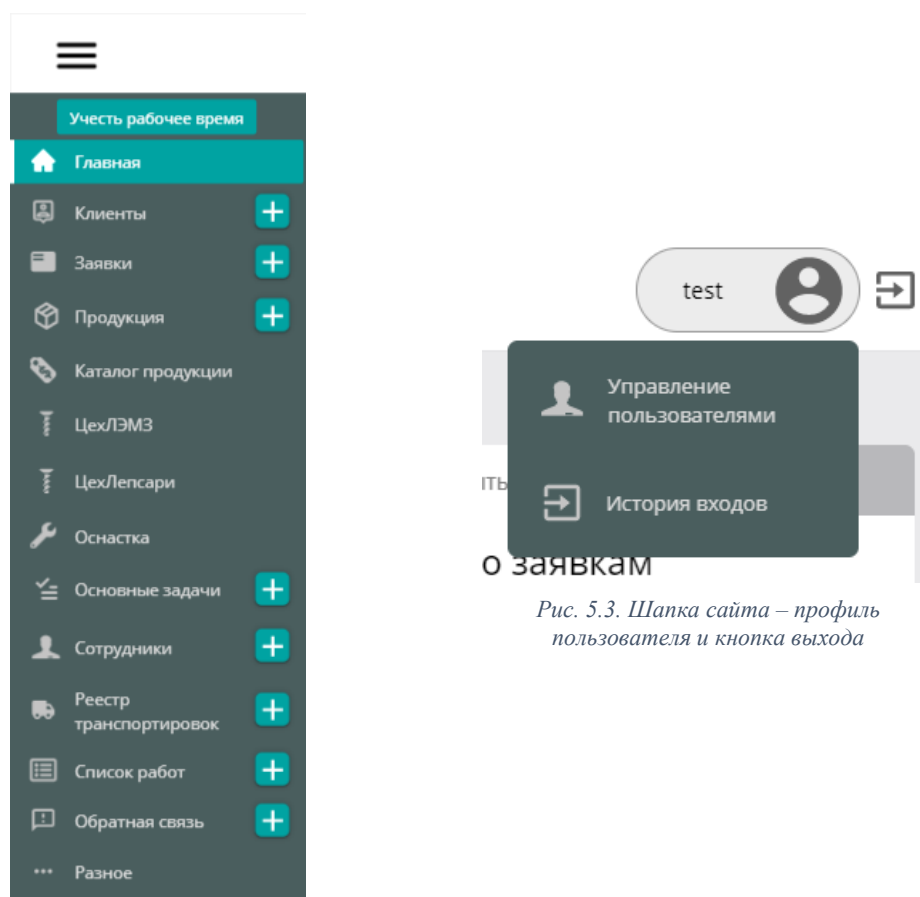


Рис. 5.3. Шапка сайта – профиль пользователя и кнопка выхода

Рис. 5.2. Боковое меню сайта с ссылками на страницы

Далее будут приведены основные элементы управления данными, которые пользователь может встретить при просмотре страниц клиентского приложения.

При взаимодействии с практически любой таблицей данных на страницах просмотра данных пользователь сможет отсортировать данные по доступным параметрам с помощью панели сортировки данных (Рис. 5.4).

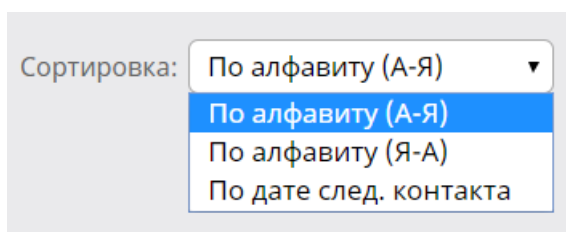


Рис. 5.4. Панель сортировки данных

На страницах просмотра таблиц данных, у которых есть определенные статусы, пользователю доступна панель выбора элементов для фильтрации данных (Рис. 5.5). При нажатии на один из элементов этой панели, данные в таблице будут отфильтрованы таким образом, что отображаться будут только элементы с указанным статусом.

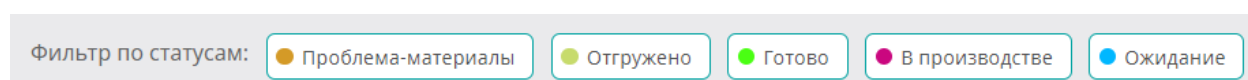


Рис. 5.5. Панель выбора элементов для фильтрации данных

Для сложных данных с различными типами, на странице просмотра предусмотрена панель вкладок (Рис. 5.6) для возможности перехода на страницы с соответствующими типами выводимых данных.



Рис. 5.6. Панель вкладок

6. Тестирование

6.1. Выбор видов тестирования

Для тестирования программного продукта существует огромное количество видов тестов, для каждого из которых есть своя область применения. Важно грамотно выбрать метод тестирования для наилучших результатов в нахождении ошибок в программе, так как самая главная цель при тестировании – найти как можно больше ошибок, для их последующего анализа и исправления.

При разработке клиентского приложения было решено использовать методы функционального и модульного тестирования.

Функциональные требования определяют, что именно делает ПО, какие задачи оно решает [5].

Модульное тестирование позволяет проверить на работоспособность отдельные модули программы.

6.2. Результаты тестирования

В результате проведения функционального и модульного методов тестирования были выявлены и исправлены следующие ошибки:

- Лишние отрисовки компонентов, которые загружали данные с сервера и обрабатывали их несколько раз.
- Асинхронность некоторых операций, хотя для их корректной работы необходимо синхронное, последовательное выполнение.
- Некорректное заполнение форм данные.

6.3. Примеры созданных тестов

Были составлены следующие тесты для модульного тестирования, на примере класса Clients и FormUser.

Класс Clients

- Успешная отрисовка компонента без ошибок – идет проверка, был ли вообще отрисован компонент.
- Успешная отрисовка самого списка с историей.
- `matchSnapshot()` – совпадает ли отрисованный компонент со снэпшотом. Снэпшот это хранимая в отдельной папке копия компонента, который должен совпадать с отрисованным.
- Проверка на то, что список по умолчанию скрыт, а затем после эмуляции нажатия на шапку – раскрыт. Сначала отрисовывается компонент, находится элемент списка и эмитируется нажатие пользователем.
- Успешное обновление компонента при изменении передаваемых в него данных. Делается `render` с новыми передаваемыми значениями.
- Успешная отрисовка, даже если не передано никаких данных.

- Успешная отрисовка, даже если типы данных не соответствуют требуемым.

```
PASS src/components/Clients/__test__/Clients.test.js
✓ renders w/o crashing (43ms)
✓ renders list correctly (17ms)
✓ matches snapshot (38ms)
✓ first list is hidden, then opens up when header is clicked (56ms)
✓ works correctly when props are updated through Main component (24ms)
✓ works correctly when no props are given (5ms)
✓ works correctly when prop types are incorrect (8ms)
```

Рис. 6.1 Выполнение тестов для класса Clients

Класс FormUser

- Выводится сообщение об ошибке при корректных входных параметрах – пользователь регистрируется.
- Выводится сообщение “Ошибка! Пароли не совпадают” при несовпадении введенных паролей.
- Выводится сообщение “Ошибка! Пароль введен некорректно” если пароль не удовлетворяет регулярному выражению, описанному в коде (минимум 8 символов, одна заглавная буква, одна маленькая буква, одна цифра).
- Выводится сообщение “Ошибка! Некорректный адрес почты” если почта, введенная пользователем не удовлетворяет регулярному выражению.

```
PASS src/components/Profile/Users/__test__/FormUser.test.js
✓ renders w/o crashing (21ms)
✓ matches snapshot (13ms)
✓ outputs success with correct inputs (30ms)
✓ outputs correct error with different passwords (19ms)
✓ outputs correct error with incorrect password (6ms)
✓ outputs correct error with incorrect email address (9ms)
```

Рис. 6.2 Выполнение тестов для класса FormUser

Заключение

В ходе дипломной работы было спроектировано и разработано клиентское приложение, являющимся частью системы управления малым предприятием.

Требования, предъявленные к данному программному обеспечению, выполнены. Получен опыт проектирования и разработки клиентского приложения, программирования на языке JavaScript с использованием библиотеки React, тестирования клиентского приложения и разработки интерфейсов.

Были реализованы следующие поставленные задачи:

- Авторизация пользователя и использование токена доступа для обеспечения безопасности
- Возможность просматривать данные о клиентах, создавать, редактировать удалять их.
- Возможность просматривать данные о заявках, создавать, редактировать удалять их.
- Возможность просматривать данные о продукции, создавать, редактировать удалять их.
- Возможность просматривать данные об обратной связи с сотрудниками предприятия, создавать, редактировать удалять их, вести обсуждения.
- Разные уровни доступа к страницам клиентского приложения в зависимости от роли пользователя.

В данный момент клиентское приложение активно используется сотрудниками предприятия, включая работу менеджера с клиентами и их заказами, диспетчера с учетом данных в реестре транспортировок, базе данных сотрудников и заказами цехов, сотрудников предприятия с учетом рабочего времени, цехов с заказами на продукцию, находящимися на очереди на производство. В ближайшем будущем планируется добавить функционал интерактивного табеля с учтенным рабочим временем сотрудников, различные графики для просмотра эффективности реализации произведенной продукции, а также усовершенствовать код программы и улучшать ее быстродействие.

Тестирование программного обеспечения помогло заранее обнаружить и исправить ошибки в клиентском приложении, тем самым уменьшив время разработки и увеличив безопасность кода и его работоспособность.

В результате выполнения данной работы удалось разработать и внедрить программное обеспечение, которое также планируется сопровождать и развивать далее.

Список Литературы

1. Официальная документация React. [Электронный ресурс]. Режим доступа: <https://reactjs.org/docs>
2. Категории субъектов малого и среднего бизнеса : Федеральный закон "О развитии малого и среднего предпринимательства в Российской Федерации" от 24.07.2007 N 209-ФЗ : принят Государственной Думой 6 июля 2007 г.
3. Organizing React Application. [Электронный ресурс] / Gaspar Nagy. Режим доступа: <https://levelup.gitconnected.com/enterprise-size-react-application-directory-structure-90b0ebc60d59>
4. Patterns for data fetching in React. [Электронный ресурс] / Gigi Sayfan. Режим доступа: <https://blog.logrocket.com/patterns-for-data-fetching-in-react-981ced7e5c56/>
5. Функциональное тестирование. [Электронный ресурс]. Режим доступа: https://ru.wikipedia.org/wiki/Функциональное_тестирование
6. Справочник по JavaScript. [Электронный ресурс]. Режим доступа: <https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference>
7. Fetch-запросы. Справочник. [Электронный ресурс]. Режим доступа: <https://learn.javascript.ru/fetch>
8. Современные ERP-системы на Российском рынке: сравнительный обзор. [Электронный ресурс] / И.С. Костарев. Режим доступа: <http://integral-russia.ru/2018/01/23/sovremennye-erp-sistemy-na-rossijskom-rynke-sravnitelnyj-obzor/>

Приложение

App.js

```
import React, { Suspense, lazy } from 'react'
import { BrowserRouter, Switch, Route } from 'react-router-dom'
import './App.scss'
import './variables.scss'
const MainPage = lazy(() => import('./components/MainPage/MainPage.jsx'))
//lazy-загрузка компонента MainPage
import LoginPage from './components/Authorization/LoginPage/LoginPage.jsx'
import PrivateRoute from './components/PrivateRoute/PrivateRoute.jsx'
import { login, refreshToken } from './utils/RequestsAPI/Authorization.jsx'
,
import PageLoading from './components/MainPage/PageLoading/PageLoading.jsx'
,
export const UserContext = React.createContext()

class App extends React.Component {
  state = {
    isAuthorized: false, //Авторизован ли пользователь
    userData: {
      //Данные пользователя
      email: '',
      username: '',
      firstName: '',
      lastName: '',
      roles: [],
      id: 0,
    },
  },
}

//Метод для обновления состояния данных пользователя
setUserData = (isAuthorized, userData) => {
  this.setState({
    isAuthorized: isAuthorized,
    userData: userData,
  })
}

//Метод для проверки на принадлежность пользователя
//к одной из ролей из переданного массива ролей
userHasAccess = (rolesNeeded) => {
  let check = false
  this.state.userData.roles.map((item) => {
    rolesNeeded.map((role) => {
      if (item.name === role) {
        check = true
      }
    })
  })
}
```

```
    return check
  }

  componentDidMount() {
    //Проверка на наличие в localStorage браузера токена,
    //запрос на его обновление при отсутствии
    if (
      localStorage.getItem('refreshToken') &&
      this.state.isAuthorized === false
    ) {
      const refreshTokenObject = Object.assign({
        refreshToken: localStorage.getItem('refreshToken'),
      })
      refreshToken(refreshTokenObject)
        .then((res) => res.json())
        .then((response) => {
          this.setUserData(true, response.user)
          localStorage.setItem('accessToken', response.accessToken)
          localStorage.setItem('refreshToken', response.refreshToken)
        })
        .catch((error) => {
          console.log(error)
          localStorage.removeItem('accessToken')
          localStorage.removeItem('refreshToken')
          window.location.reload()
        })
    }
  }

  render() {
    return (
      <BrowserRouter>
        <Switch>
          <Route
            path="/login"
            render={ (props) => (
              <LoginPage
                isAuthorized={this.state.isAuthorized}
                setUserData={this.setUserData}
                {...props}
              />
            ) }
          />
          { /* Отображение компонента загрузки страницы, пока грузятся внут
ренние компоненты */ }
          <Suspense fallback={<PageLoading />}>
            <UserContext.Provider
              value={{
                userData: { ...this.state.userData },
                userHasAccess: this.userHasAccess,
              }}
            </UserContext.Provider>
          </Suspense>
        </Switch>
      </BrowserRouter>
    )
  }
}
```

```
        >
        <PrivateRoute
          path="/"
          isAuthorized={this.state.isAuthorized}
          userData={this.state.userData}
          component={MainPage}
          userHasAccess={this.userHasAccess}
        />
      </UserContext.Provider>
    </Suspense>
  </Switch>
</BrowserRouter>
)
}
}

export default App
```

productsAPI.js

```
import { request } from '../utilsAPI.jsx'

//GET-запрос всех записей из таблицы продукции
export function getProducts() {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/',
    method: 'GET',
  })
}

//GET-запрос для получения записи продукции по ее id
export function getProductById(id, signal) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/' + id,
    method: 'GET',
    signal: signal,
  })
}

//POST-
запрос для получения записей продукции по названию категории продукции
export function getProductsByCategory(category, signal) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/category/',
    method: 'POST',
    body: JSON.stringify(category),
    signal: signal,
  })
}

//POST-запрос для получения записей продукции
```

```
//по названию подразделения предприятия, на котором она производится
export function getProductsByLocation(workshop, signal) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/location/',
    method: 'POST',
    body: JSON.stringify(workshop),
    signal: signal,
  })
}

//DELETE-запрос для удаления продукции по ее id
export function deleteProduct(id) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/' + id,
    method: 'DELETE',
  })
}

//POST-запрос для добавление записи продукции в таблицу
export function addProduct(newProduct) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/',
    method: 'POST',
    body: JSON.stringify(newProduct),
  })
}

//PUT-запрос для изменение записи продукции в таблице
export function editProduct(newProduct, id) {
  return request({
    url: process.env.API_BASE_URL + '/api/v1/product/' + id,
    method: 'PUT',
    body: JSON.stringify(newProduct),
  })
}
```

FileUploader.jsx

```
import React, { useState } from 'react'
import fileImg from '../../assets/file.svg'
import './FileUploader.scss'

const FileUploader = (props) => {
  //Строка для хранения названия загруженного файла
  const [imgName, setImgName] = useState('Имя файла...')

  return (
    <div className="file-uploader">
      <div className="file-uploader__file_name">{imgName}</div>
    </div>
  )
}
```

```
<label className="file-uploader__label" htmlFor={'file' + props.uniqueId}>
  Загрузить файл
  <img className="file-uploader__img" src={fileImg} alt="" />
</label>
<input
  type="file"
  name="file"
  id={'file' + props.uniqueId}
  onChange={(event) => {
    //При загрузке файла, проверяем удовлетворяет ли файл
    //необходимому формату
    let file = event.target.files[0]
    if (file.name.match(props.regex) !== null) {
      setImgName(file.name) //Устанавливаем имя файла
      let reader = new FileReader()
      reader.onloadend = () => {
        props.onChange(reader.result)
      }
      //Для разных типов файла - разные функции обработки данных
      switch (props.type) {
        default:
          reader.readAsDataURL(file)
          break
        case 'readAsArrayBuffer':
          reader.readAsArrayBuffer(file)
      }
    } else {
      setImgName('Некорректный формат файла!')
    }
  }}
/>
<div
  className="file-uploader__remove"
  onClick={() => {
    setImgName('Имя файла...')
    props.onChange('')
  }}
>
  <div className="file-uploader__line"></div>
  <div className="file-uploader__line"></div>
</div>
</div>
)
}
```

export default FileUploader

Requests.jsx

```
import React, { useState, useEffect } from 'react'
import '../../../../utils/MainWindow/MainWindow.scss'
import './Requests.scss'
import '../../../../utils/Form/Form.scss'
import '../../../../utils/MainWindow/MainWindow.scss'
import {
  getRequests,
  deleteRequest,
  deleteProductsToRequest,
  getRequestById,
} from '../../../../utils/RequestsAPI/Requests.jsx'
import TableView from './TableView/TableView.jsx'
import SearchBar from './SearchBar/SearchBar.jsx'
import FormWindow from '../../../../utils/Form/FormWindow/FormWindow.jsx'

const Requests = (props) => {
  const [requests, setRequests] = useState([]) //Массив заявок
  const [searchQuery, setSearchQuery] = useState('') //Значение строки поиска
  const [showWindow, setShowWindow] = useState(false) //Показывать ли окно
  const [isLoading, setIsLoading] = useState(false) //Индикатор загрузки
  const [toWorkshop, setToWorkshop] = useState('lemz') //Название цеха для переноса заявки
  //id заявки, использующийся при ее дальнейшем копировании или переносе в цеха
  const [requestId, setRequestId] = useState(0)
  const [clients, setClients] = useState([]) //Массив клиентов
  const [curPage, setCurPage] = useState('Открытые') //Текущая страница
  //Статусы заявок
  const [requestStatuses, setRequestStatutes] = useState([
    {
      name: 'Проблема/Материалы',
      oldName: 'Проблема-материалы',
      className: 'materials',
      access: ['ROLE_ADMIN', 'ROLE_WORKSHOP'],
      visible: false,
    },
    {
      name: 'Отгружено',
      className: 'shipped',
      access: ['ROLE_ADMIN', 'ROLE_WORKSHOP'],
      visible: false,
    },
    {
      name: 'Готово к отгрузке',
      oldName: 'Готово',
      className: 'ready',
      access: ['ROLE_ADMIN', 'ROLE_MANAGER'],
      visible: false,
    },
  ],
  {

```

```
    name: 'В производстве',
    className: 'in-production',
    access: ['ROLE_ADMIN', 'ROLE_MANAGER', 'ROLE_MANAGER'],
    visible: false,
  },
  {
    name: 'Ожидание',
    className: 'waiting',
    access: ['ROLE_ADMIN', 'ROLE_MANAGER', 'ROLE_MANAGER'],
    visible: false,
  },
])

//Удалить заявку
const deleteItem = (event) => {
  const id = event.target.dataset.id
  getRequestById(id)
    .then((res) => res.json())
    .then((res) => {
      const productsArr = res.requestProducts.map((product) => {
        return deleteProductsToRequest(product.id)
      })
      Promise.all(productsArr).then(() => {
        deleteRequest(id).then(() => loadRequests())
      })
    })
}

useEffect(() => {
  document.title = 'Заявки'
  let abortController = new AbortController()
  loadRequests(abortController.signal)
  return function cancel() {
    abortController.abort()
  }
}, [curPage])

//Получение списка клиентов из массива заявок
const getClientsFromRequests = (reqs) => {
  const temp = clients
  reqs.map((request) => {
    if (
      request.status !== 'Завершено' &&
      !temp.find((item) => request.codeWord === item.name)
    ) {
      temp.push({
        name: request.codeWord,
        active: false,
      })
    }
  })
}
```

```
// console.log(temp);
setClients([...temp])
}

//GET-запрос на получение всех заявок
const loadRequests = (signal) => {
  setIsLoading(true)
  getRequests(signal)
    .then((res) => res.json())
    .then((requests) => {
      let temp = requests.map((item) => {
        return {
          ...item,
          open: false,
        }
      })
      setIsLoading(false)
      console.log(temp)
      setRequests(temp)
      getClientsFromRequests(temp)
    })
    .catch((error) => {
      setIsLoading(false)
      console.log(error)
    })
}

//Перенести заявку
const transferRequest = (id) => {
  setRequestId(id)
  setShowWindow(!showWindow)
}

//Копировать заявку
const copyRequest = (id) => {
  props.setTransferState(true)
  props.setTransferData(
    requests.find((item) => {
      if (item.id === id) {
        return true
      }
    })
  ),
  props.history.push('/requests/new')
}

return (
  <div className="requests">
    <div className="main-window">
      <div className="main-window__header">
        <div className="main-window__title">Заявки</div>

```

```
<SearchBar
  title="Поиск по заявкам"
  placeholder="Введите название продукции для поиска..."
  setSearchQuery={setSearchQuery}
/>
<div className="main-window__menu">
  <div
    className={
      curPage === 'Открытые'
        ? 'main-window__item--active main-window__item'
        : 'main-window__item'
    }
    onClick={() => setCurPage('Открытые')}
  >
    Открытые
  </div>
  <div
    className={
      curPage === 'Завершено'
        ? 'main-window__item--active main-window__item'
        : 'main-window__item'
    }
    onClick={() => setCurPage('Завершено')}
  >
    Завершено
  </div>
</div>
<FormWindow
  title="Перенос заявки в план производства"
  windowName="transfer-request"
  content={
    <React.Fragment>
      <div className="main-form">
        <div className="main-form__form">
          <div className="main-form__item">
            <div className="main-
form__input_name">Подразделение</div>
            <div className="main-form__input_field">
              <select
                name="workshop"
                onChange={(event) => {
                  setToWorkshop(event.target.value)
                }}
              >
                <option value="lemz">ЦехЛЭМЗ</option>
                <option value="lepsari">ЦехЛепсари</option>
              </select>
            </div>
          </div>
          <div className="main-form__buttons">
```

```
        <input
          className="main-form__submit"
          type="submit"
          onClick={() => {
            props.setTransferState(true)
            props.setTransferData(
              requests.find((item) => {
                if (item.id === requestId) {
                  return true
                }
              })
            ),
            props.history.push(
              toWorkshop + '/workshop-' + toWorkshop + '/new',
            )
          }}
          value="Перенести в цех"
        />
      </div>
    </div>
  </div>
</React.Fragment>
}
showWindow={showWindow}
setShowWindow={setShowWindow}
/>
<div className="main-window__info-panel">
  <div className="requests__clients-sort">
    {clients
      .sort((a, b) => {
        if (a.name < b.name) {
          return -1
        }
        if (a.name > b.name) {
          return 1
        }
        return 0
      })
      .map((client, index) => {
        return (
          <div
            className={
              (client.active
                ? 'main-window__button'
                : 'main-window__button main-window__button--
inverted') +
              ' main-window__list-item--' +
              status.className
            }
            onClick={() => {
              let temp = clients
```

```

        temp = temp.map((item) => {
          return {
            ...item,
            active: false,
          }
        })
        temp.splice(index, 1, {
          ...client,
          active: !client.active,
        })
        setClients([...temp])
      }}
    >
    {client.name}
  </div>
)
)}}
</div>
<div className="main-window__amount_table">
  Всеро: {requests.length} записей
</div>
</div>
<div className="main-window__status-panel">
  <div>Фильтр по статусам: </div>
  {requestStatuses.map((status, index) => {
    return (
      <div
        className={
          (status.visible
            ? 'main-window__button'
            : 'main-window__button main-window__button--
inverted') +
          ' main-window__list-item--' +
          status.className
        }
        onClick={() => {
          let temp = requestStatuses.map((status) => {
            return {
              ...status,
              visible: false,
            }
          })
          temp.splice(index, 1, {
            ...status,
            visible: !status.visible,
          })
          setRequestStatutes([...temp])
        }}
      >
      {status.name}
    </div>
  )}
)

```

```
    )
  }}}
</div>
<TableView
  data={requests
    .filter((item) => {
      if (curPage === 'Открытые') {
        if (item.status !== 'Завершено') return true
      } else {
        if (item.status === 'Завершено') return true
      }
    })
    .filter((item) => {
      let check = false
      let noActiveStatuses = true
      requestStatuses.map((status) => {
        requestStatuses.map((status) => {
          if (status.visible) {
            noActiveStatuses = false
          }
        })
      })
      if (
        noActiveStatuses === true ||
        (status.visible &&
          (status.name === item.status ||
            status.oldName === item.status))
      ) {
        check = true
        return
      }
    })
    return check
  })
    .filter((item) => {
      let check = false
      let noActiveClients = true
      clients.map((client) => {
        if (client.active) {
          noActiveClients = false
        }
      })
    })
    clients.map((client) => {
      if (
        noActiveClients ||
        (client.active && client.name === item.codeWord) ||
        item.status === 'Завершено'
      ) {
        check = true
        return
      }
    })
  })
```

```
        return check
      })}
      isLoading={isLoading}
      loadData={loadRequests}
      deleteItem={deleteItem}
      transferRequest={transferRequest}
      copyRequest={copyRequest}
      searchQuery={searchQuery}
      userHasAccess={props.userHasAccess}
    />
  </div>
</div>
)
}

export default Requests
```

PrivateRoute.jsx

```
import React from 'react'
import { Redirect, Route } from 'react-router-dom'
import NotAllowedPage from '../MainPage/NotAllowedPage/NotAllowedPage.jsx'

const PrivateRoute = ({
  component: Component,
  isAuthorized,
  userHasAccess,
  allowedRoles,
  userData,
  ...rest
}) => (
  //Если пользователь не авторизован и у него нет токена
  //доступа, то редирект на /login, иначе - рендер
  //передаваемого компонента
  <Route
    {...rest}
    render={ (props) =>
      localStorage.getItem('refreshToken') ? (
        allowedRoles !== undefined ? (
          userHasAccess(allowedRoles) ? (
            <Component
              {...rest}
              {...props}
              userData={userData}
              userHasAccess={userHasAccess}
            />
          ) : (
            <NotAllowedPage />
          )
        )
      ) : (
        <NotAllowedPage />
      )
    }
  />
)
```

```
    ) : (
      <Component
        {...rest}
        {...props}
        userData={userData}
        userHasAccess={userHasAccess}
      />
    )
  ) : (
    <Redirect
      to={{
        pathname: '/login',
        state: { from: props.location },
      }}
    />
  )
}
/>
)
```

```
export default PrivateRoute
```

NewClient.jsx

```
import React, { useState, useEffect } from 'react'
import './NewClient.scss'
import '../../../utils/Form/Form.scss'
import { addClient } from '../../../utils/RequestsAPI/Clients.jsx'
import { addClientLegalEntity } from '../../../utils/RequestsAPI/Clients/LegalEntity.jsx'
import { addClientContact } from '../../../utils/RequestsAPI/Clients/Contacts.jsx'
import { addClientWorkHistory } from '../../../utils/RequestsAPI/Clients/WorkHistory.jsx'
import SelectLegalEntity from '../../../SelectLegalEntity/SelectLegalEntity.jsx'
import InputText from '../../../utils/Form/InputText/InputText.jsx'
import InputDate from '../../../utils/Form/InputDate/InputDate.jsx'
import ErrorMessage from '../../../utils/Form/ErrorMessage/ErrorMessage.jsx'
import SelectContacts from '../../../SelectContacts/SelectContacts.jsx'
import SelectClientCategory from '../../../ClientCategories/SelectClientCategory/SelectClientCategory.jsx'
import SelectWorkHistory from '../../../SelectWorkHistory/SelectWorkHistory.jsx'
import InputUser from '../../../utils/Form/InputUser/InputUser.jsx'
import { getUsers } from '../../../utils/RequestsAPI/Users.jsx'
import Button from '../../../utils/Form/Button/Button.jsx'

const newClient = (props) => {
  const [clientInputs, setClientInputs] = useState({
    name: '',
```

```
legalEntity: [],
contacts: [],
managerName: props.userData.username,
managerId: props.userData.id,
workHistory: [],
site: '',
comment: '',
storageAddress: '',
workCondition: '',
price: '',
discount: '',
check: '',
clientType: 'Активные',
categoryId: 0,
categoryName: '',
nextContactDate: new Date(new Date().setDate(new Date().getDate() + 7)
), //Прибавляем 7 дней к сегодняшнему числу
}))
//Ошибки для обязательных полей
const [formErrors, setFormErrors] = useState({
  name: false,
  contacts: false,
  categoryId: false,
  site: false,
})
//Корректность заполнения обязательных полей
const [validInputs, setValidInputs] = useState({
  name: false,
  contacts: false,
  categoryId: false,
  site: false,
})

const [showError, setShowError] = useState(false)
const [isLoading, setIsLoading] = useState(false)
const [users, setUsers] = useState([])
const [curTab, setCurTab] = useState('clientData')

//Проверка поля на корректность
const validateField = (fieldName, value) => {
  switch (fieldName) {
    default:
      if (validInputs[fieldName] !== undefined) {
        setValidInputs({
          ...validInputs,
          [fieldName]: value !== '',
        })
      }
      break
  }
}
```

```
//Проверка всех полей формы на корректность
const formIsValid = () => {
  let check = true
  let newErrors = Object.assign({
    name: false,
    contacts: false,
    site: false,
  })
  for (let item in validInputs) {
    // console.log(item, validInputs[item]);
    if (validInputs[item] === false) {
      check = false
      newErrors = Object.assign({
        ...newErrors,
        [item]: true,
      })
    }
  }
  setFormErrors(newErrors)
  if (check === true) {
    return true
  } else {
    // alert("Форма не заполнена");
    setIsLoading(false)
    setShowError(true)
    return false
  }
}

//Добавление клиента и остальных обязательных данных
//при корректном заполнении всей формы
const handleSubmit = () => {
  setIsLoading(true)
  console.log(clientInputs)
  let clientId = 0
  formIsValid() &&
  addClient({
    clientType: clientInputs.clientType,
    comment: clientInputs.comment,
    manager: clientInputs.managerName,
    name: clientInputs.name,
    price: clientInputs.price,
    site: clientInputs.site,
    storageAddress: clientInputs.storageAddress,
    workCondition: clientInputs.workCondition,
    check: clientInputs.check,
    nextDateContact: clientInputs.nextContactDate.getTime() / 1000,
    categoryId: clientInputs.categoryId,
  })
  .then((res) => res.json())
}
```

```
.then((res) => {
  clientId = res.id
  return Promise.all(
    clientInputs.legalEntity.map((item) => {
      return addClientLegalEntity({
        name: item.name,
        inn: item.inn,
        kpp: item.kpp,
        ogrn: item.ogrn,
        bik: item.bik,
        checkingAccount: item.checkingAccount,
        legalAddress: item.legalAddress,
        factualAddress: item.factualAddress,
        legalEntity: item.legalEntity,
        clientId: res.id,
      })
    })
  )
})
.then(() => {
  return Promise.all(
    clientInputs.contacts.map((item) => {
      return addClientContact({
        name: item.name,
        lastName: item.lastName,
        email: item.email,
        position: item.position,
        phoneNumber: item.phoneNumber,
        clientId: clientId,
      })
    })
  )
})
.then(() => {
  return addClientWorkHistory({
    date: new Date(),
    action: 'Создание записи',
    result: 'Запись успешно создана',
    comment: '<Сообщение сгенерировано автоматически>',
    clientId: clientId,
  })
})
.then(() => {
  return Promise.all(
    clientInputs.workHistory.map((item) => {
      return addClientWorkHistory({
        date: item.date,
        action: item.action,
        result: item.result,
        comment: item.comment,
        clientId: clientId,
      })
    })
  )
})
```

```
        ))
      }),
    )
  })
  .then(() => {
    return props.history.push(
      '/clients/category/' + clientInputs.categoryName + '/active',
    )
  })
  .catch((error) => {
    setIsLoading(false)
    alert('Ошибка при добавлении записи')
    console.log(error)
  })
}
```

//При изменении данных в поле - обновляем состояние

```
const handleInputChange = (e) => {
  const { name, value } = e.target
  validateField(name, value)
  setClientInputs({
    ...clientInputs,
    [name]: value,
  })
  setFormErrors({
    ...formErrors,
    [name]: false,
  })
}
```

```
useEffect(() => {
  document.title = 'Добавление клиента'
  //Загружаем список пользователей
  users.length === 0 &&
  getUsers()
    .then((res) => res.json())
    .then((res) => {
      const filteredRoles = ['ROLE_ADMIN', 'ROLE_MANAGER']
      let newUsers = res
      setUsers([
        ...newUsers
          .filter((item) => {
            const temp = filteredRoles
            ? item.roles.reduce((prevRole, curRole) => {
              let check = false
              filteredRoles.map((filteredRole) => {
                if (filteredRole === curRole.name) {
                  check = true
                  return
                }
              })
            })
          })
      ])
    })
})
```

```
        return check
      }, false)
    : true
    return temp
  })
  .map((item) => {
    return {
      ...item,
      active: true,
    }
  }),
])
})
}, [users])

return (
  <div className="new_client">
    <div className="main-form">
      <div className="main-form__title">Новый клиент</div>
      { /* //Меню для перехода на разные страницы формы */ }
      <div className="main-form__header">
        <div
          className={
            curTab === 'workHistory'
              ? 'main-form__menu-item main-form__menu-item--active'
              : 'main-form__menu-item'
          }
          onClick={() => {
            setCurTab('workHistory')
          }}
        >
          История работы
        </div>
        <div
          className={
            curTab === 'clientData'
              ? 'main-form__menu-item main-form__menu-item--active'
              : 'main-form__menu-item'
          }
          onClick={() => {
            setCurTab('clientData')
          }}
        >
          Данные клиента
        </div>
      </div>
      <form className="main-form__form">
        <ErrorMessage
          message="Не заполнены все обязательные поля!"
          showError={showError}
          setShowError={setShowError}
        />
      </form>
    </div>
  </div>
)
```

```
</>
{curTab === 'workHistory' ? (
  <React.Fragment>
    {/* Добавление истории работ */}
    <div className="main-form__item">
      <div className="main-
form__input_name">История работ*</div>
      <div className="main-form__input_field">
        <SelectWorkHistory
          handleWorkHistoryChange={(value) => {
            validateField('workHistory', value)
            setClientInputs({
              ...clientInputs,
              workHistory: value,
            })
          }}
          defaultValue={clientInputs.workHistory}
          userHasAccess={props.userHasAccess}
        />
      </div>
    </div>
  </React.Fragment>
) : (
  <React.Fragment>
    <InputText
      inputName="Название"
      required
      name="name"
      error={formErrors.name}
      defaultValue={clientInputs.name}
      errorsArr={formErrors}
      setErrorsArr={setFormErrors}
      handleInputChange={handleInputChange}
    />
    {/* Добавление юридических лиц */}
    <div className="main-form__item">
      <div className="main-
form__input_name">Юридическое лицо</div>
      <div className="main-form__input_field">
        <SelectLegalEntity
          handleLegalEntityChange={(value) => {
            // validateField("legalEntity", value);
            setClientInputs({
              ...clientInputs,
              legalEntity: value,
            })
          }}
          defaultValue={clientInputs.legalEntity}
          userHasAccess={props.userHasAccess}
        />
      </div>
    </div>
  </React.Fragment>
)
```

```
    </div>
    <div className="main-form__fieldset">
      <div className="main-form__group-
name">Контактные данные</div>
      { /* Добавление контактных лиц */ }
      <div className="main-form__item">
        <div className="main-
form__input_name">Контактное лицо*</div>
        <div className="main-form__input_field">
          <SelectContacts
            handleContactsChange={ (value) => {
              validateField('contacts', value)
              setClientInputs({
                ...clientInputs,
                contacts: value,
              })
            }}
            defaultValue={clientInputs.contacts}
            userHasAccess={props.userHasAccess}
          />
        </div>
      </div>
      <div>
        <InputText
          inputName="Сайт"
          required
          name="site"
          error={formErrors.site}
          errorsArr={formErrors}
          setErrorsArr={setFormErrors}
          defaultValue={clientInputs.site}
          handleInputChange={handleInputChange}
        />
      </div>
      <InputText
        inputName="Комментарий"
        name="comment"
        defaultValue={clientInputs.comment}
        handleInputChange={handleInputChange}
        type="textarea"
      />
      <InputText
        inputName="Адрес склада"
        name="storageAddress"
        defaultValue={clientInputs.storageAddress}
        handleInputChange={handleInputChange}
      />
      <InputDate
        inputName="Дата след. контакта"
        name="nextContactDate"
        selected={Date.parse(clientInputs.nextContactDate)}
        handleDateChange={ (value) => {
```

```
        setClientInputs({
          ...clientInputs,
          nextContactDate: value,
        })
      })
    }
  }
</>
<InputText
  inputName="Условия работы"
  name="workCondition"
  defaultValue={clientInputs.workCondition}
  handleInputChange={handleInputChange}
/>
<InputText
  inputName="Прайс"
  name="price"
  defaultValue={clientInputs.price}
  handleInputChange={handleInputChange}
/>
<InputUser
  inputName="Ответственный менеджер"
  userData={props.userData}
  filteredRoles={['ROLE_ADMIN', 'ROLE_MANAGER']}
  required
  name="manager"
  handleUserChange={(value, id) => {
    setClientInputs({
      ...clientInputs,
      managerId: Number.parseInt(id),
      managerName: value,
    })
  }}
  defaultValue={clientInputs.managerName}
  searchPlaceholder="Введите имя менеджера для поиска..."
/>
<InputText
  inputName="Акт сверки"
  name="check"
  handleInputChange={handleInputChange}
  defaultValue={clientInputs.check}
/>
<div className="main-form__fieldset">
  <div className="main-form__group-
name">Категория клиента</div>
  <div className="main-form__item">
    <div className="main-
form__input_name">Тип клиента*</div>
    <div className="main-form__input_field">
      <select
        name="clientType"
        onChange={handleInputChange}
        defaultValue={clientInputs.clientType}
```

```
        >
        <option value="Активные">Активные</option>
        <option value="Потенциальные">Потенциальные</option>
        <option value="В разработке">В разработке</option>
      </select>
    </div>
  </div>
  <SelectClientCategory
    inputName="Выбор категории клиента"
    required
    error={formErrors.category}
    userHasAccess={props.userHasAccess}
    windowName="select-category"
    name="categoryId"
    handleCategoryChange={(value, name) => {
      validateField('categoryId', value)
      setClientInputs({
        ...clientInputs,
        categoryId: value,
        categoryName: name,
      })
      setFormErrors({
        ...formErrors,
        categoryId: false,
      })
    }}
    defaultValue={clientInputs.categoryName}
    errorsArr={formErrors}
    setErrorsArr={setFormErrors}
  />
</div>
</React.Fragment>
)}
<div className="main-form__input_hint">
  * - поля, обязательные для заполнения
</div>
<div className="main-form__buttons">
  <input
    className="main-form__submit main-form__submit--inverted"
    type="submit"
    onClick={() => props.history.push('/clients/categories')}
    value="Вернуться назад"
  />
  <Button
    text="Добавить клиента"
    isLoading={isLoading}
    className="main-form__submit"
    onClick={handleSubmit}
  />
</div>
</form>
```

```
        </div>
      </div>
    )
  }

export default newClient
```

GeneralTasks.jsx

```
import React, { useEffect, useState } from 'react'
import './GeneralTasks.scss'
import '../../../utils/MainWindow/MainWindow.scss'
import SearchBar from '../../../SearchBar/SearchBar.jsx'
import TableView from './TableView/TableView.jsx'
import {
  getMainTasks,
  deleteMainTask,
} from '../../../utils/RequestsAPI/MainTasks.jsx'

const GeneralTasks = (props) => {
  const [generalTasks, setGeneralTasks] = useState([])
  const [searchQuery, setSearchQuery] = useState('')
  const [isLoading, setIsLoading] = useState(false)
  //Статусы задач
  const [taskStatuses, setTaskStatuses] = useState([
    {
      name: 'Выполнено',
      className: 'completed',
      visible: false,
    },
    {
      name: 'Отложено',
      className: 'delayed',
      visible: false,
    },
    {
      name: 'Материалы',
      className: 'materials',
      visible: false,
    },
    {
      name: 'В процессе',
      className: 'in-progress',
      visible: false,
    },
    {
      name: 'Проблема',
      className: 'problem',
      visible: false,
    },
  ],
```

```
  ])  
  
  useEffect(() => {  
    document.title = 'Основные задачи'  
    //abortController для прерывания загрузки данных  
    //при переходе на другие страницы  
    let abortController = new AbortController()  
    loadTasks(abortController.signal)  
    return function cancel() {  
      abortController.abort()  
    }  
  }, [])  
  
  //Запрос на получение списка задач  
  const loadTasks = (signal) => {  
    setIsLoading(true)  
    return getMainTasks(signal)  
      .then((res) => res.json())  
      .then((res) => {  
        // console.log(res);  
        setIsLoading(false)  
        setGeneralTasks(res)  
      })  
      .catch((error) => {  
        //Обработка ошибок  
        setIsLoading(false)  
        console.log(error)  
      })  
  }  
  
  //Удаление задачи  
  const deleteItem = (id) => {  
    deleteMainTask(id).then(() => loadTasks())  
  }  
  
  return (  
    <div className="general_tasks">  
      <div className="main-window">  
        <div className="main-window__title">Основные задачи</div>  
        <SearchBar  
          title="Поиск по задачам"  
          placeholder="Введите описание задачи для поиска..."  
          setSearchQuery={setSearchQuery}  
        />  
        <div className="main-window__info-panel">  
          <div className="main-window__status-panel">  
            <div>Фильтр по статусам: </div>  
            {taskStatuses.map((status, index) => {  
              return (  
                <div  
                  className={
```

```

        (status.visible
        ? 'main-window__button'
        : 'main-window__button main-window__button--
inverted') +
        ' main-window__list-item--' +
        status.className
    }
    onClick={() => {
        let temp = taskStatuses.map((status) => {
            return {
                ...status,
                visible: false,
            }
        })
        temp.splice(index, 1, {
            ...status,
            visible: !status.visible,
        })
        setTaskStatuses([...temp])
    }}
    >
        {status.name}
    </div>
    )
    })}
</div>
<div className="main-window__amount_table">
    Всего: {generalTasks.length} записей
</div>
</div>
<TableView
    data={generalTasks.filter((item) => {
        let check = false
        let noActiveStatuses = true
        taskStatuses.map((status) => {
            taskStatuses.map((status) => {
                if (status.visible) {
                    noActiveStatuses = false
                }
            })
        })
        if (
            noActiveStatuses === true ||
            (status.visible && status.name === item.condition)
        ) {
            check = true
            return
        }
    })}
    return check
    })}
    })}
searchQuery={searchQuery}
```

```
        userData={props.userData}
        userHasAccess={props.userHasAccess}
        deleteItem={deleteItem}
        isLoading={isLoading}
        taskStatuses={taskStatuses}
        loadData={loadTasks}
      />
    </div>
  </div>
)
}
```

```
export default GeneralTasks
```

functions.jsx

```
//Получение строки типа 'дд.мм.ГГГГ' из объекта Date
export const formatDateString = (dateString) => {
  // const testDate = new Date(Date.parse(dateString));
  const temp = new Date(dateString)
  const testDate = new Date(
    temp.getFullYear() + '/' + (temp.getMonth() + 1) + '/' + temp.getDate(
  ),
  )
  return (
    (testDate.getDate() < 10 ? '0' + testDate.getDate() : testDate.getDate
  ()) +
    '.' +
    (testDate.getMonth() + 1 < 10
      ? '0' + (testDate.getMonth() + 1)
      : testDate.getMonth() + 1) +
    '.' +
    testDate.getFullYear()
  )
}

export const formatDateStringNoYear = (dateString) => {
  // console.log(dateString);
  const temp = new Date(dateString)
  const testDate = new Date(
    temp.getFullYear() + '/' + (temp.getMonth() + 1) + '/' + temp.getDate(
  ),
  )
  return (
    (testDate.getDate() < 10 ? '0' + testDate.getDate() : testDate.getDate
  ()) +
    '.' +
    (testDate.getMonth() + 1 < 10
      ? '0' + (testDate.getMonth() + 1)
      : testDate.getMonth() + 1)
  )
}
```

```
}

export const formatDateStringWithTime = (dateString) => {
  const testDate = new Date(dateString)
  return (
    (testDate.getDate() < 10 ? '0' + testDate.getDate() : testDate.getDate()) +
    '.' +
    (testDate.getMonth() + 1 < 10
      ? '0' + (testDate.getMonth() + 1)
      : testDate.getMonth() + 1) +
    '.' +
    testDate.getFullYear() +
    ' ' +
    (testDate.getHours() < 10
      ? '0' + testDate.getHours()
      : testDate.getHours()) +
    ':' +
    (testDate.getMinutes() < 10
      ? '0' + testDate.getMinutes()
      : testDate.getMinutes())
  )
}

export const formatDateStringToTime = (dateString) => {
  const testDate = new Date(dateString)
  return (
    (testDate.getHours() < 10
      ? '0' + testDate.getHours()
      : testDate.getHours()) +
    ':' +
    (testDate.getMinutes() < 10
      ? '0' + testDate.getMinutes()
      : testDate.getMinutes())
  )
}

//Определение склонения слова в зависимости от остатка числа
//к которому оно относится
export const numberToString = (number, wordForms) => {
  number = Math.abs(number) % 100
  let remainder = number % 10
  if (number > 10 && number < 20) {
    return wordForms[2]
  }
  if (remainder > 1 && remainder < 5) {
    return wordForms[1]
  }
  if (remainder == 1) {
    return wordForms[0]
  }
}
```

```
    return wordForms[2]
  }

  //Функция для скачивания переданной картинки
  export const imgToBlobDownload = (imageSrc, imageName) => {
    var img = new Image()
    img.src = imageSrc
    var c = document.createElement('canvas')
    var ctx = c.getContext('2d')
    c.width = img.naturalWidth
    c.height = img.naturalHeight
    ctx.drawImage(img, 0, 0)
    c.toBlob(
      function (blob) {
        // получаем содержимое как JPEG Blob
        let link = document.createElement('a')
        link.download = imageName
        link.href = URL.createObjectURL(blob)
        link.click()
        // удаляем ссылку на Blo
        URL.revokeObjectURL(link.href)
      },
      'image/jpeg',
      1,
    )
    img.crossOrigin = '' // if from different origin
    img.src = 'url-to-image'
  }

  //Получение URI из полученной картинки
  export function getDataUri(url, extension, quality) {
    return new Promise((resolve, reject) => {
      var img = new Image()
      img.onload = () => {
        var canvas = document.createElement('canvas')
        canvas.width = img.width
        canvas.height = img.height
        var ctx = canvas.getContext('2d')
        ctx.drawImage(img, 0, 0)
        var dataURL = canvas.toDataURL(
          extension ? 'image/' + extension : 'image/png',
          quality ? quality : 0.92,
        )
        resolve(dataURL)
      }
      img.onerror = (error) => {
        reject(error)
      }
      img.src = url
    })
  }
}
```

```
//Функция для изменения порядка сортировки
export const changeSortOrder = (event) => {
  const name = event.target.value.split(' ')[0]
  const order = event.target.value.split(' ')[1]
  return Object.assign({
    curSort: name,
    [name]: order,
  })
}

//Получение списка работ сотрудников из полученного массива
export const getAllProductsFromWorkCount = (works) => {
  let parts = {}
  works.map((work) => {
    // console.log(work)
    work.workControlProduct.map((product) => {
      if (parts[product.product.id] === undefined) {
        return (parts = Object.assign({
          ...parts,
          [product.product.id]: {
            name: product.product.name,
            quantity: Number.parseFloat(product.quantity),
          },
        )))
      } else {
        return (parts = Object.assign({
          ...parts,
          [product.product.id]: {
            ...parts[product.product.id],
            quantity:
              Number.parseFloat(parts[product.product.id].quantity) +
              Number.parseFloat(product.quantity),
          },
        )))
      }
    })
  })
  // console.log(parts)
  return parts
}

//Получить случайный цвет формата hex
export const getRandomColor = () => {
  var letters = '0123456789ABCDEF'
  var color = '#'
  for (var i = 0; i < 6; i++) {
    color += letters[Math.floor(Math.random() * 16)]
  }
  return color
}
```

```
//Сложить два цвета
export const addHexColor = (color1, color2) => {
  var hexStr = (parseInt(color1, 16) + parseInt(color2, 16)).toString(16)
  while (hexStr.length < 6) {
    hexStr = '0' + hexStr
  }
  return hexStr
}

//Добавить разделитель пробел для полученной строки
export const addSpaceDelimiter = (str) => {
  return str.toString().replace(/(\d)(?=(\d\d\d)+([^\d]|$))/g, '$1 ')
}
```